Professional AI Engineering & Modern Agentic AI Development Training

Facilitator: Ramkumar J Dhamodharan
https://www.linkedin.com/in/ramkumar-j-d-57423611

Email: jd.ramkumar@gmail.com
Phone: +91-98860-82730

To prepare your own Lab Setup

Clone the repository

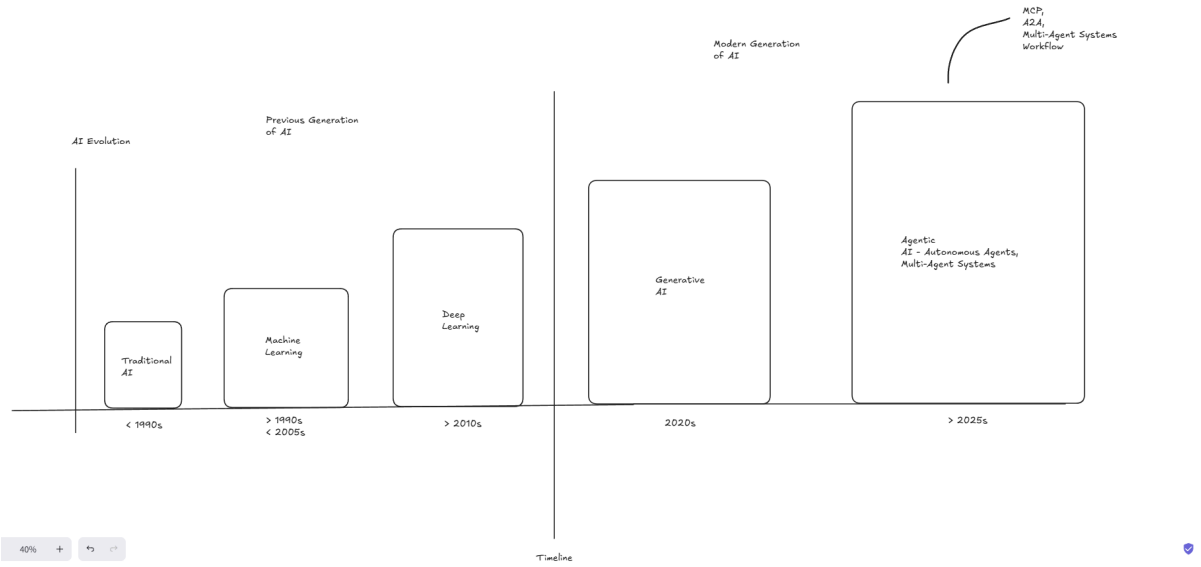
https://github.com/iomegak12/lab-systems

And execute install-lab-software.ps1

Virtual Delivery - Habits of All Days

- Lecture - 20% - Demonstrations / Practices - 80%
- You shall more focus onto learning concepts and how to build solutions with AI
    - ML Model Design | Development | Deployment (Manual)
        - AI Driven
- Business Use Case with Several Problem Statements and Solving with Technical Implementations
- All Source codes will be available in GitHub Repositories, which you can refer after the training as well
- Training Notes shall be saved as PDF and shared through GitHub Repositories

AI Evolution - Quick Summary



Traditional AI

- Rule Based Systems
- Complex | Mega Decisions Trees
- Manually Crafted Conditions
    - If-Else-Then-Else-End If

Data Mining - A process of finding insights within the data by applying mathematical | statistical techniques

Machine Learning

- Data Driven Learning
- How the data is correlated | relationships | patterns within the data
    - By applying Statistics | Mathematics fields (Algebra | Calculus)

Deep Learning

- Neural Networks based Data Driven Learning

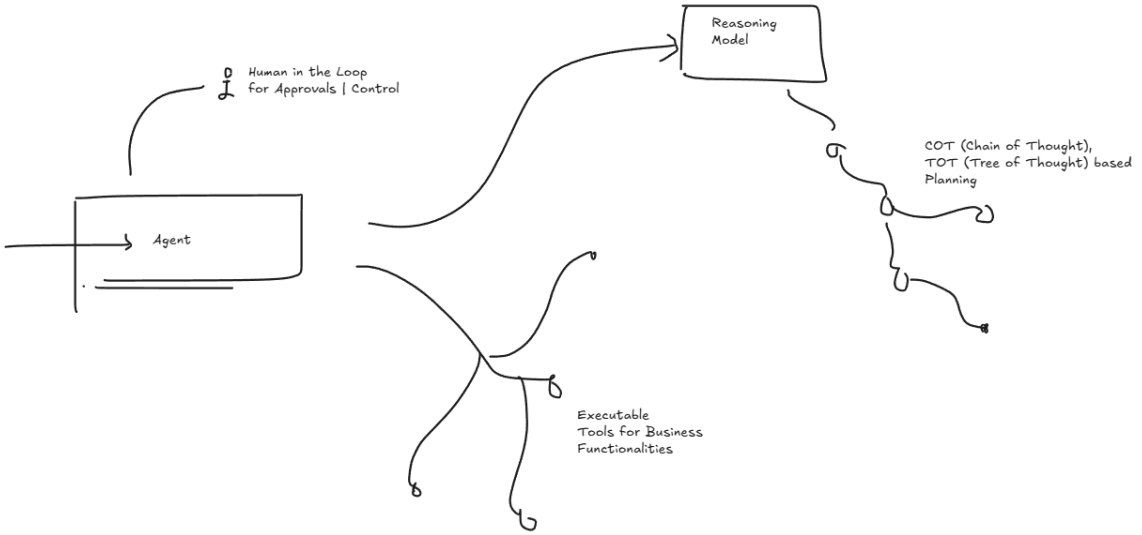
Advancements of DL -> Generative AI

- Deep Learning Models that have NL* capabilities - Language Model
- NL*
    - NLU
    - NLP
    - NLR

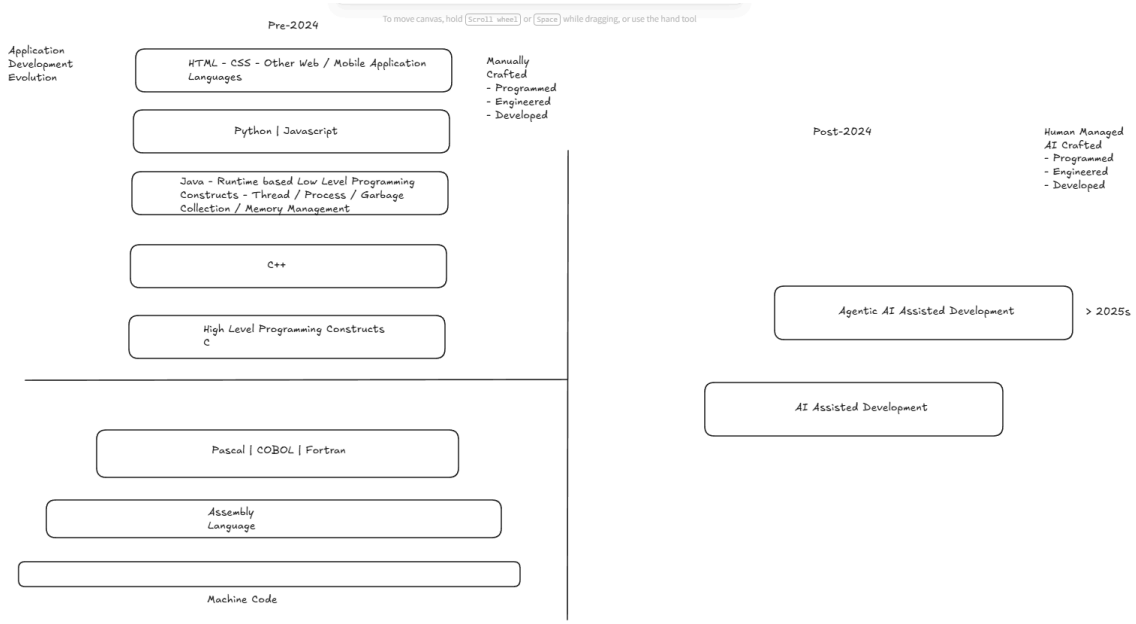
Agentic AI

- Agent
    - A program which has thinking abilities with the help of LLM + Reasoning and has the ability to take actions with the help of provided tools

How does an Agent work?



Application Development Evolution



My Simple Experience based Motto

- You trust that AI can do anything related to your IT work &
- But, Don't trust AI that it has done the work

Python Foundations

- A programming language to simplify how programs are written (Like the way you speak)
- Instead of being descriptive, It's designed to be expressive
- It's an OO Programming Language | Procedural Programming Supported
- Interpreted
- Community | Enterprise Adoption in Data Science World
- C/C++ libraries are compatible
- 100% - Data Science specific problem solvers | Libraries | SDKs available
- PyPI
    - Package Repositories for Reusable Python Libraries
- VENV
    - Virtual Environment
    - Isolated Execution Environment

How do you set up a typical environment for Python Application Development?

- In a folder, you create and activate a virtual environment
- By default, VENV is a Python Module, using which virtual environments can be provisioned, managed and activated/deactivated.
- In Modern days, to simplify the PyPI package references, Virtual Environment Management and Python Version Management, it's strongly recommended to use **UV** (A specialized Python Tool for VENV, Package and Version Management)
- In Python, package means a set of reusable libraries, functionalities that solve common problems such as IO, Networking, Storage, Database and so on.
- Most of the business use cases, you should be looking for a ready-made / prebuilt package in PyPI repository, Community or you build a customized package
- Before you build your applications in Python, it's strongly recommended that you experiment your thoughts / ideas with Jupyter Notebooks
- VS Code is preferred editor in Modern Application Development for Python
- A typical Python application shall have the following files
    - requirements.txt - this file is mainly used for describing package dependencies that your application depends on
        - requirements.prod.txt - requirements.dev.txt
    - uv.lock (this file is mainly used for Python Package Manager especially a package, it's dependencies with version details & repositories)
    - Every folder in a Python Application is a module, and which can be referenced with a relative path notation
        - It's strongly advised to have __init__.py file which describes what features of the module exposed (For example, a module may have 10s classes, out of which, what to be exposed to the user)
    - README.md - describes the purpose of the application
    - LICENSE (Any specific License Information)
    - .gitignore (This file is mainly used for Version control system what to be included / excluded)
    - Other files are optional - CODE OF CONDUCT, SECURITY, TROUBLESHOOTING, CHANGELOG and CONTRIBUTING.md (markdown files)

Verify the Lab Environment

> In the cloud lab, Click Docker Desktop and make sure that it's running

Once the Docker Desktop is up and running,

> Click Terminal

> cd lab-systems

> pwsh .\installation-check.ps1

NOTE: You should be able to see all checks are passed. If Kubernetes specific warning comes up or SSMS failure - both can be IGNORED.

> in the terminal

> md c:\demonstrations\firststeps

> cd c:\demonstrations\firststeps

> uv init –python=3.12
(two dashes)

> uv sync

> .venv\scripts\activate

Now, you should see your folder name is prefixed with () - it indicates that the virtual environment is activated

> uv run main.py

> notepad main.py

> paste the following contents

```
def main():
    print("Hello from firststeps!")


if __name__ == "__main__":
    main()
```

> save the file
> in the terminal, uv run main.py

> once you have tested, in the terminal,

> code .

> this should open the current python application folder in vs code for further development

> in vs code -> view -> command palette

> select for Python Interpreter

> select python interpreter

> you should be able to by default see all virtual environments available

> select firststeps virtual environment

> create a folder named notebooks

> create a file named “1-first-steps.ipynb”

> click the button select kernel (top right)

> select virtual environments -> select the favorite (star marked) virtual environment

> click code

> paste

print(“Welcome to the World Python Virtual Environment and Jupyter!!!”)

> click the execute cell button

> If it’s prompting for ipykernel package is to be installed, install it

> you should now see the output

Additional Steps:

> if you want to install Python and Jupyter extensions in VS code

> left navigation -> extensions

> search for python

> select python from Microsoft

> search for Jupyter

> select Jupyter from Microsoft

Variables, Dictionaries, Array, Accessing Elements, Performing Calculations with Loops

> create a notebook named 2-essetials.ipynb and select the kernel

> click + code

```
import csv
import json

from datetime import date
from pathlib import Path
```

> click + code

```
subscriptions = {
    "subscription_id": "SUB128908",
    "account_name": "Apex Cloud",
    "product": "Cornava Collaborate",
    "tier": "Essentials",
    "seats": 120,
    "acv_usd": 23000.74,
    "auto_renew": False
}

term = (date(2026, 10, 1), date(2027, 10, 1))
products = ["Cornava Collaborate",
            "Cornava SecureBridge", "Cornava NetOps Cloud"]
regions_seen = {"Americas", "EMEA", "APJC", "EMEA"}
```

> click + code

```
print(type(subscriptions["seats"]).__name__)
print(type(subscriptions["acv_usd"]).__name__)
print(type(subscriptions["auto_renew"]).__name__)
```

> another code

```
print("ACV Per Seat : ", round(subscriptions["acv_usd"] /
subscriptions["seats"]))
```

> + code

```
print("Unique Regions: ", sorted(regions_seen))
print("How many Number of Regions: ", len(regions_seen))
```

> + code

```
short_product_names = [product.replace("Cornava ", "").upper() for
product in products]

print(short_product_names)
```

> + code

```
print("Term Length (in Days) : ", (term[1] - term[0]).days)
```

Business Use Case - Demonstration & Practices

Fictional Company: Corvana Networks
Business Problem Statement: Subscription Renewals

Product Portfolio - Collaborate, SecureEdge, NetOps Cloud and Observe
Every product is sold in three different tiers - Essentials, Advantage and Premier

Customer Accounts (accounts.csv)

Day 1 - Business Question: What does our Renewal book actually look like?

- Clear the CRM Pipeline Extract
- Profile Accounts and Usage
- Chart and Summarize Renewal Health

Capabilities:

- Python Programming
- NumPy
- Pandas
- Matplotlib | Seaborn
- Statistics

Data Assets:

> accounts.csv - One Row per customer account
> subscriptionc.csv - One Row per subscription (account * product)
> monthly_usage.csv - One Row per subscription per month
> renewal_pipeline_extract.txt - Raw pipe delimited CRM export of upcoming renewals with defects
> pricing_rules.json - Price List, Tier subscriptions and Discount Bands

Prepare the use case in the lab

> In terminal

> Open a new tab

> cd c:\demonstrations

> git clone https://github.com/iomegak12/aiml-day1

> cd aiml-day1

> uv init –python=3.12
(two dashes)

> uv sync

> uv add -r requirements.txt

> code .

> vs code -> view -> command palette -> select python interpreter to Virtual Environment specific Python Runtime

> create a folder named “notebooks”

> create a file named step-1.ipynb and select the kernel (right top)

> choose virtual environments -> choose workspace specific virtual environment

> add a code

```
from pathlib import Path

DATA = Path("../data")
OUT = Path("./output")

OUT.mkdir(exist_ok=True)

for file in sorted(DATA.iterdir()):
    print(f"{file.name} {file.stat().st_size / 1024:8.1f} KB")
```

> run the cell

> click + code

```
from datetime import date

import csv
import json

AS_OF = date(2026, 9, 30)
```

> execute the cell

> click + code

```
with open(DATA/"pricing_rules.json") as file:
    pricing = json.load(file)

print(list(pricing.keys()))
```

> click + code

```
def quote_acv(product, tier, seats, segment, channel="Direct"):
    """
    Return the Annual Contract Value (USD) for a quote, using
pricing_rules.json
    """

    if seats <= 0:
        raise ValueError(f"Seats must be positive, got : {seats}")

    list_price = pricing["list_price_per_seat_per_year"][product] * \
        pricing["tier_multiplier"][tier]
    discount = pricing["segment_discount_pct"][segment]

    if channel == "Partner":
        discount += pricing["partner_extra_discount_pct"]

    discount = min(discount, pricing["max_discount_pct"])
    return round(seats * list_price * (1-discount/100), 2)
```

> click + code

```
print(quote_acv("Corvana SecureEdge", "Advantage", 500, "Mid-Market"))
```

```
print(quote_acv("Corvana SecureEdge", "Advantage", 500, "Mid-Market",
channel= "Partner"))
```

> click + code

```
try:
    quote_acv("Corvana Observe", "Premier", 0, "SMB")
except ValueError as error:
    print("Value Error : ", error)

try:
    quote_acv("Corvana Voice", "Premier", 10, "SMB")
except KeyError as error:
    print("Key Error : Unknown Product", error)
```

> + code

```
def urgency(renewal_date, as_of=AS_OF):
    days = (renewal_date - as_of).days

    if days < 0:
        return "Overdue"
    elif days <= 30:
        return "Critical"
    elif days <= 90:
        return "Upcoming"

    return "Later"
```

> + code

```
urgency_result = urgency(date(2026, 9, 1))

print(urgency_result)
```

> + Code

```
clean = []
rejects = []
header = None

with open(DATA/"renewal_pipeline_extract.txt") as file:
    for line_no, line in enumerate(file, start=1):
        line = line.strip()

        if not line or line.startswith("#"):
            continue

        fields = line.split("|")

        if header is None:
            header = fields
            continue

        try:
            if len(fields) != len(header):
                raise ValueError(f"Expected {len(header)} fields, got :
{len(fields)}")

            row = dict(zip(header, fields))
            row["seats"] = int(row["seats"])

            if row["seats"] <= 0:
                raise ValueError(f"Seats must be positive, got :
{row['seats']}")

            row["acv_usd"] = float(row["acv_usd"])
            row["renewal_date"] =
date.fromisoformat(row["renewal_date"])

            clean.append(row)
        except ValueError as error:
```

```
            rejects.append((line_no, str(error)))

print(f"Clean rows : {len(clean)}")
print(f"Rejected rows : {len(rejects)}")
```

> + code

```
with open(OUT/"renewal_pipeline_clean.csv", "w", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=header)
    writer.writeheader()
    writer.writerows(clean)

with open(OUT/"renewal_pipeline_errors.log", "w") as file:
    for line_no, reason in rejects:
        file.write(f"Line {line_no} : {reason}\n")

print(f"Wrote {len(clean)} rows to {OUT/'renewal_pipeline_clean.csv'}")
print(f"Wrote {len(rejects)} rejects to
{OUT/'renewal_pipeline_errors.log'}")
```

Prompt Used

can you update notebooks\step-1.ipynb file, to add the following functionality at the end

- Read the file line (data\renewal_pipeline_extract.txt) by line with open and enumerate
- skip blank lines and lines starting with # - The first remaining line is the header
- For every other line, inside a try block
a. check it has the same number of fields as the header
b. convert seats to int and reject zero or negative values
c. convert acv_usd to float
d. convert renewal_date to date with iso format
- good rows go into a list clean and bad rows go into rejects with line number and reason
- write output\renewal_pipeline_clean.csv
- and output\rewnewal_pipeline_errors.log with one line per reject

do you understand my requirements? if you have any questions, clarify with me.

Data Libraries - NumPy and Pandas

NumPy

- A library which helps to perform numerical operations in Python
    - Especially array of values
- A NumPy array is a block of same-typed numbers in contiguous memory
- That's what makes it fast, and every ML library expects
- NumPy arrays hold raw numbers in ONE DTYPE
- Operations run in compiled C - 100x faster
- Common vocabularies we use in NumPy
    - Scalar (One Number)
    - Vector (1-D array of values)
    - Matrix (2-D array of values)
    - Tensor (3-D+ - array of values)
- Indexing and Slicing
    - Array[0] - Indexing
    - Array[:, -3:] - Slicing
- Fancy Indexing
    - WIth a list of positions and boolean indexing
    - Array[attribute1 <= 0.5]
    - Array[0,5,10,14]
- Slices are views, NOT COPIES
    - Changing a slice changes the original
    - Use always copy() when you mean it
- Common Numerical Operations in NumPy
    - Vectorized Operations
    - To avoid looping by ourselves
    - Broadcasting
    - Reshaping
    - Aggregation
    - Linear Algebra Basics
- Vectorization
    - Active / seats divides every element without a loop
- Reshaping
    - A mechanism of changing the dimensions from rows -> columns, columns -> rows
- Broadcasting
    - Virtually dividing them into multiple groups, so all | partially can be processed in parallel
- Aggregation
    - Axis Property
        - 0 - Rows
        - 1 - Columns
- Linear Algebra
    - @ is matrix multiplication
    - . - transpose

Pandas

- It’s a library for Tabular Data Science
- Library for Data Processing | Wrangling | Munging
- Series
    - A 1-D labelled array (one column)
- Data Frame
    - A table of series sharing an index
- Creating a Data Frame - from a dictionary of lists, from a list of dictionaries, often from a file (CSV, JSON)
- Common functions
        > shape
        > head
        > dtypes
        > info
        > describe
    > SQL | Excel - Operations to Pandas
        > Selecting Columns
        > Boolean Filter
        > GroupBy
        > merge
        > pviot_table
- Pandas holds everything in-memory
- Alternatives to Pandas for Complex | Big Data Processing
    - Polars
    - Spark
    - SQL Engine
- Common Operations in Pandas
    - Filtering
    - Grouping
    - Aggregations
    - Joins
    - Merging

Common Libraries for Analytics

- NumPy
- Pandas
- Matplotlib
- Seaborn
- SciPy (Scientific Operations)
- Scikit (ML)

Hands-On Practices with NumPy

> in vs code -> create a new file named 2-NumPy.ipynb and select the kernel

```
from pathlib import Path

import numpy as np

DATA = Path("../data")
np.set_printoptions(precision=3, suppress=True)
```

>

```
NUMERIC_COLUMNS = ["seats_purchased", "active_users",
"feature_adoption_score", "support_tickets"]

with open(DATA/"monthly_usage.csv") as file:
    header = file.readline().strip().split(",")

usage = np.loadtxt(
    DATA/"monthly_usage.csv",
    delimiter=",",
    skiprows=1,
    usecols=[header.index(column) for column in NUMERIC_COLUMNS],
)

subscription_ids = np.loadtxt(
    DATA/"monthly_usage.csv",
    delimiter=",",
    skiprows=1,
    usecols=header.index("subscription_id"),
    dtype=str,
)
```

```
for name, array in [("usage", usage), ("subscription_ids",
subscription_ids)]:
    print(f"{name}")
    print(f"  shape : {array.shape}")
    print(f"  ndim  : {array.ndim}")
    print(f"  dtype : {array.dtype}")
    print(f"  size  : {array.size}")

print(f"\nColumns : {NUMERIC_COLUMNS}")
print("First 3 rows :")
print(usage[:3])
print(subscription_ids[:3])
```

>

```
MONTHS = 12

active_users = usage[:,
NUMERIC_COLUMNS.index("active_users")].reshape(-1, MONTHS)
support_tickets = usage[:,
NUMERIC_COLUMNS.index("support_tickets")].reshape(-1, MONTHS)

seats = usage[::MONTHS, NUMERIC_COLUMNS.index("seats_purchased")]
sub_ids = subscription_ids[::MONTHS]

seats_by_month = usage[:,
NUMERIC_COLUMNS.index("seats_purchased")].reshape(-1, MONTHS)
seats_constant = np.all(seats_by_month == seats[:, np.newaxis])

print(f"active_users    : {active_users.shape}")
print(f"support_tickets : {support_tickets.shape}")
print(f"seats           : {seats.shape}")
print(f"sub_ids         : {sub_ids.shape}")
print(f"\nSeats constant within each subscription : {seats_constant}")

print(f"\n{sub_ids[0]} seats : {seats[0]}")
print(f"active_users    : {active_users[0]}")
print(f"support_tickets : {support_tickets[0]}")
```

>

```
print(f"First subscription, all 12 months ({sub_ids[0]}) :")
print(active_users[0])

print(f"\nLast 3 months of the first two subscriptions {sub_ids[:2]}
:")
print(active_users[:2, -3:])

print(f"\nMonth 6 of subscription index 10 ({sub_ids[10]}) :")
print(active_users[10, 5])

picks = [0, 10, 100]

print(f"\nMonth 1 for subscriptions {picks} {sub_ids[picks]} :")
print(active_users[picks, 0])


>

utilization = active_users / seats[:, np.newaxis]

fleet_by_month = utilization.mean(axis=0)
per_subscription = utilization.mean(axis=1)

print(f"Fleet average utilization per month (axis=0)
{fleet_by_month.shape} :")
print(fleet_by_month)

print(f"\nPer subscription average utilization (axis=1)
{per_subscription.shape} :")
print(f"  min  : {per_subscription.min():.3f}")
print(f"  max  : {per_subscription.max():.3f}")
print(f"  mean : {per_subscription.mean():.3f}")

busiest = active_users[0].argmax()

print(f"\nBusiest month for subscription 0 ({sub_ids[0]}) : month
{busiest + 1} with {active_users[0, busiest]:.0f} active users")
print(f"Total tickets for subscription 0 :
{support_tickets[0].sum():.0f}")
print(f"Total tickets across the fleet : {support_tickets.sum():.0f}")


>
```

```
recent_utilization = utilization[:, -3:].mean(axis=1)
low_utilization = recent_utilization < 0.5

print(f"Average utilization over the last 3 months
{recent_utilization.shape} :")
print(recent_utilization[:5])

print(f"\nMask {low_utilization.shape} {low_utilization.dtype} :")
print(low_utilization[:5])

print(f"\nSubscriptions below 50% : {low_utilization.sum()} of
{low_utilization.size} ({low_utilization.mean():.1%})")
print(f"First 5 IDs : {sub_ids[low_utilization][:5]}")
print(f"Their utilization : {recent_utilization[low_utilization][:5]}")
```

Prompts used:

can you refer and update notebooks\step-2.ipynb notebook for the following requirements
- load the numerical columns (seats_purchased, active_users, feature_adoption_score, support_tickets) with NumPy, plus the subscription_id column as text

Print shape, ndim, dtype and size and the first 3 rows

the file has one row per subscription per month, sorted by subscription the month. so literally every 12 rows is ONE subscription.

what I want you to do is that ...

reshape active users and support tickets into subscriptions, 12 matrices. Take seats and IDs from every 12th row. Verify that the seats are constant within each subscription.

NOTE: update the notebook at the bottom

do you understand my requirement?

Now, i want to try Indexing and Slicing
- print the first subscriptions' 12 months
- the last 3 months of the first two subscriptions
- the month-6 value of subscription index 10
- month 1 for subscriptions 0
- 10 and 100 using fancy indexing

do you get my requirements?

Now, I want to try NumPy aggregations
- fleet average utilization per month (axis=0)
- per subscription average - min, max and mean
- business month for subscription 0 and total tickets

do you get my points?

Now, I'm going to try Boolean Masking

- computer each subscriptions' average utilization over the last 3 months
- count those below 50%

Pandas - Demonstration and Practices

> in vs code -> create a new file named 3-pands.ipynb and select the kernel

> + code

```
from pathlib import Path

import pandas as pd

DATA = Path("../data")
OUTPUT = Path("../output")
OUTPUT.mkdir(exist_ok=True)

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)
```

>

```
accounts = pd.read_csv(DATA/"accounts.csv",
parse_dates=["customer_since"])
subscriptions = pd.read_csv(DATA/"subscriptions.csv",
parse_dates=["start_date", "end_date"])
monthly_usage = pd.read_csv(DATA/"monthly_usage.csv",
parse_dates=["month"], date_format="%Y-%m")

frames = {"accounts": accounts, "subscriptions": subscriptions,
"monthly_usage": monthly_usage}

for name, frame in frames.items():
    print(f"{name:<14}: {frame.shape}")

print("\nParsed date columns :")
for name, frame in frames.items():
    for column, dtype in
frame.select_dtypes("datetime").dtypes.items():
        label = f"{name}.{column}"
        print(f"  {label:<25}: {dtype}")

print("\nColumn dtypes in subscriptions :")
print(subscriptions.dtypes.value_counts().to_string())
```

>

```
for name, frame in frames.items():
    print(f"{name} : first 3 rows")
    display(frame.head(3))
```

>

```
KEY_COLUMNS = {
    "accounts": ["account_id", "account_name", "segment", "region",
"customer_since"],
    "subscriptions": ["subscription_id", "account_id", "product",
"annual_contract_value", "end_date", "renewal_status"],
    "monthly_usage": ["subscription_id", "month", "seats_purchased",
"active_users", "seat_utilization"],
}

for name, frame in frames.items():
    print(f"{name} : key columns")
    display(frame[KEY_COLUMNS[name]].head(3))
```

>

```
acv = subscriptions["annual_contract_value"]

print(f"type   : {type(acv)}")
print(f"length : {len(acv)}")
print(f"sum    : {acv.sum():,.2f}")
print(f"max    : {acv.max():,.2f}")
```

>

```
print("Renewal statuses :")
print(subscriptions["renewal_status"].value_counts().to_string())

print("\nSegment shares :")
print(accounts["segment"].value_counts(normalize=True).to_string())
```

>

```
merged = subscriptions.merge(accounts, on="account_id", how="left",
validate="many_to_one", indicator=True)
```

```
missing_account = (merged["_merge"] == "left_only").sum()
merged = merged.drop(columns="_merge")

print(f"subscriptions : {subscriptions.shape}")
print(f"accounts      : {accounts.shape}")
print(f"merged        : {merged.shape}")

print(f"\nRow count preserved : {len(merged) == len(subscriptions)}")
print(f"Subscriptions missing an account : {missing_account}")
```

>

```
by_region = (
    merged.groupby("region")
    .agg(
        subscriptions=("subscription_id", "count"),
        total_acv=("annual_contract_value", "sum"),
        median_acv=("annual_contract_value", "median"),
        avg_utilization=("avg_seat_utilization", "mean"),
    )
    .sort_values("total_acv", ascending=False)
)

print("By region, sorted by total ACV :")
display(by_region.style.format({"total_acv": "${:,.2f}", "median_acv":
"${:,.2f}", "avg_utilization": "{:.3f}"}))
```

>

```
acv_pivot = merged.pivot_table(
    index="segment",
    columns="product",
    values="annual_contract_value",
    aggfunc="sum",
    fill_value=0,
)

print(f"Total ACV by segment and product {acv_pivot.shape} :")
display(acv_pivot.style.format("${:,.2f}"))
```

Prompts

can you refer and update the notebook file
C:\000-AIML\demonstrations\aiml-day1\notebooks\step-3.ipynb for the following features

- load the three CSVs (accounts.csv, monthly_usage.csv and subscriptions.csv)
- parse the date columns properly
- print each shape, count the column dtypes in subscriptions
- and show first 3 rows for each pandas data frame
- few key columns also to be displayed

take the ACV column as a series, print it's type, length, sum and max.

count renewal statuses, and show segment shares with normalization enabled

i want to try merge subscriptions with accounts
- left merge subscriptions with accounts on account id
- validate many to one relationship is honored
- check the shape and no subscription is missing an account

let's try grouping, aggregation and pivot operations
- by region - number of subscriptions, total ACV, median ACV and average utilization, sorted by total ACV
- a pivot table of total ACV by segment and product
- and can you add currency representation to the view

Day 2 Agenda

- Summarize Day 1 Contents & Learnings
- Matplotlib and Seaborn - Introduction & Demonstration
- (L) ML Foundations
- (L) ML Development Lifecycle
- (L) Scikit - Learn - Introduction, Features & How does it work?
- (L) EDA - Introduction
- First ML Model
- EDA - Demonstration & Practices

Matplotlib and Seaborn - Libraries for Data Visualization

- Dataviz Library
- Analysis - working on data that reveals surface level truths
- Analytics - working on data that reveals hidden truths

4 Different types of Analytics

- Descriptive Analytics - what happened?
- Diagnostic Analytics - why happened?
- Predictive Analytics - what may happen?
- Prescriptive Analytics - how do I make it happen?

In order to accomplish Descriptive and Diagnostic Analytics, you should be performing / designing charts, graphs and dashboards.

The whole idea of performing Descriptive and Diagnostic Analytics, is to explain and explore data insights.

- Explanatory Data Analytics (Charts / Graphs)
- Exploratory Data Analytics (Dashboards)

Matplotlib - The grandfather of Python Visualization. It's a low level library that offers total control over every element in a figure (lines, labels, ticks) but requires more code to look polished.

Seaborn - A high-level library built on the top of Matplotlib, integrates deeply with Pandas Dataframe and provides a beautiful, statistically default theme based with much less code.

Matplotlib - You must understand the OO structure. Every plot consists of a hierarchy.

- Figure (The overall window)
- Axes (The actual plot / subplot inside the figure)

OO Approach - instead of using plot(), explicitly define your figures and axes.

Seaborn - Simplifies complex statistical plots. You pass it a PDF (Pandas Data Frame), specify which columns belong on the X and Y axes, and use the hue semantic to map a third categorical variable to colors automatically.

Commonly used Plot Types

- Distribution (Histogram, KDE Plots)
- Categorical (Box Plot, Bar Plot, Violin Plot)
- Relationships (Scatter Plot, Line Plot)
- Matrix (Heatmap)

> In vs code -> create a file named 4-dataviz.ipynb and select the kernel

> + markdown

Learning Matplotlib and Seaborn - Dataviz Libraries

> click tick symbol to render

> code

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
> DATA = Path("../data")
OUTPUT = Path("../output")
OUTPUT.mkdir(exist_ok=True)

accounts = pd.read_csv(DATA / "accounts.csv")
subscriptions = pd.read_csv(DATA / "subscriptions.csv")
usage = pd.read_csv(DATA / "monthly_usage.csv")

df = subscriptions.merge(accounts, on="account_id")
merged_usage = usage.merge(
    df[["subscription_id", "segment"]], on="subscription_id")
closed = df[df["renewal_status"] != "Open"]
```

>

```
SEGMENT_COLORS = {
    "Enterprise": "#2a78d6",
    "Mid-Market": "#eb6834",
    "Public Sector": "#1baf7a",
    "SMB": "#eda100",
}

segment_trend = (
    merged_usage.groupby(["month", "segment"])["seat_utilization"]
    .mean()
    .unstack("segment")
)
```

```
fig, ax = plt.subplots(figsize=(10, 5))

for segment in segment_trend.columns:
    ax.plot(
        segment_trend.index,
        segment_trend[segment],
        label=segment,
        color=SEGMENT_COLORS[segment],
        linewidth=2,
        marker="o",
        markersize=4,
    )

ax.set_title("Utilization trend by segment")
ax.set_xlabel("Month")
ax.set_ylabel("Average seat utilization")
ax.yaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
ax.tick_params(axis="x", rotation=45)
ax.grid(axis="y", alpha=0.3)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(title="Segment")

fig.tight_layout()
plt.show()
```

```
>
```

```
region_acv = (
    df.groupby("region")["annual_contract_value"]
    .sum()
    .div(1_000_000)
    .sort_values(ascending=False)
)

fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(region_acv.index, region_acv.values, color="#2a78d6",
width=0.6)
ax.bar_label(bars, fmt="${:,.1f}M", padding=3)

ax.set_title("Total ACV by region")
ax.set_xlabel("Region")
```

```
ax.set_ylabel("Total ACV ($M)")
ax.margins(y=0.1)
ax.grid(axis="y", alpha=0.3)
ax.set_axisbelow(True)
ax.spines[["top", "right"]].set_visible(False)

fig.tight_layout()
plt.show()
```

>

```
TIER_COLORS = {
    "Essentials": "#2a78d6",
    "Advantage": "#eb6834",
    "Premier": "#1baf7a",
}

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

for ax, scale in zip(axes, ["linear", "log"]):
    for tier, color in TIER_COLORS.items():
        tier_rows = df[df["tier"] == tier]
        ax.scatter(
            tier_rows["seats_purchased"],
            tier_rows["annual_contract_value"],
            label=tier,
            color=color,
            s=18,
            alpha=0.6,
            edgecolors="white",
            linewidths=0.4,
        )

    ax.set_xscale(scale)
    ax.set_yscale(scale)
    ax.set_title(f"Seats vs ACV by tier : {scale} axes")
    ax.set_xlabel("Seats purchased")
    ax.set_ylabel("Annual contract value")
    ax.xaxis.set_major_formatter(lambda value, _: f"{value:,.0f}")
    ax.yaxis.set_major_formatter(lambda value, _: f"${value:,.0f}")
    ax.grid(alpha=0.3)
    ax.set_axisbelow(True)
```

```
    ax.spines[["top", "right"]].set_visible(False)
    ax.legend(title="Tier", loc="upper left")

fig.tight_layout()
plt.show()
```

>

```
STATUS_COLORS = {
    "Renewed": "#2a78d6",
    "Churned": "#eb6834",
}

fig, ax = plt.subplots(figsize=(10, 5))

sns.histplot(
    data=closed,
    x="avg_seat_utilization",
    hue="renewal_status",
    hue_order=list(STATUS_COLORS),
    palette=STATUS_COLORS,
    bins=30,
    alpha=0.5,
    edgecolor="white",
    ax=ax,
)

ax.set_title("Average seat utilization of closed renewals")
ax.set_xlabel("Average seat utilization")
ax.set_ylabel("Subscriptions")
ax.xaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
ax.get_legend().set_title("Renewal status")
sns.despine(ax=ax)

fig.tight_layout()
plt.show()
```

>

```
fig, ax = plt.subplots(figsize=(10, 5))

sns.boxplot(
    data=df,
    x="segment",
    y="annual_contract_value",
    hue="segment",
    order=list(SEGMENT_COLORS),
    palette=SEGMENT_COLORS,
    log_scale=True,
    legend=False,
    width=0.5,
    ax=ax,
)

ax.set_title("ACV by segment : log scale")
ax.set_xlabel("Segment")
ax.set_ylabel("Annual contract value")
ax.yaxis.set_major_formatter(lambda value, _: f"${value:,.0f}")
ax.grid(axis="y", alpha=0.3)
ax.set_axisbelow(True)
sns.despine(ax=ax)

fig.tight_layout()
plt.show()
```

>

```
fig, ax = plt.subplots(figsize=(8, 5))

sns.countplot(
    data=closed,
    x="tier",
    hue="renewal_status",
    order=list(TIER_COLORS),
    hue_order=list(STATUS_COLORS),
    palette=STATUS_COLORS,
    width=0.6,
    ax=ax,
)
```

```
ax.set_title("Renewal outcome by tier")
ax.set_xlabel("Tier")
ax.set_ylabel("Subscriptions")
ax.grid(axis="y", alpha=0.3)
ax.set_axisbelow(True)
ax.get_legend().set_title("Renewal status")
sns.despine(ax=ax)

fig.tight_layout()
plt.show()
```

>

```
HEALTH_METRICS = {
    "avg_seat_utilization": "Seat utilization",
    "utilization_trend": "Utilization trend",
    "avg_feature_adoption": "Feature adoption",
    "support_cases_12m": "Support cases (12m)",
    "sev1_cases_12m": "Sev1 cases (12m)",
    "nps_score": "NPS score",
}

correlations =
df[list(HEALTH_METRICS)].rename(columns=HEALTH_METRICS).corr()

fig, ax = plt.subplots(figsize=(8, 6.5))

sns.heatmap(
    correlations,
    annot=True,
    fmt=".2f",
    vmin=-1,
    vmax=1,
    cmap="vlag",
    square=True,
    linewidths=1,
    cbar_kws={"label": "Pearson correlation"},
    ax=ax,
)

ax.set_title("Correlation of health metrics")
ax.tick_params(axis="x", rotation=45)
```

```
plt.setp(ax.get_xticklabels(), ha="right")

fig.tight_layout()
plt.show()
```

Prompts used for the above Dataviz:

can you understand the notebook notebooks\step-4.ipynb, which i would like to perform a series of data visualizations using Matplotlib and Seaborn libraries.

do NOT render any charts | graphs | code | scripts at this moment. I just want you to understand.

i want to draw a line chart - utilization trend by the segment

- average seat utilization by month and segment and then ploe one line per segment

NOTE: you always add generated python code at the bottom of the notebook. For now, prefer matplotlib.

i would like to now draw a bar chart - ACV by region
your task is to generate scripts required for generating a bar chart for Total ACV by region in $M, sorted with the value printed on each bar

Now, let's draw a scatter plot - scatter seats against ACV, one color per tier, on log-log axes ... try it without log scales first and compare

Now, i would like to try it out with Seaborn, instead of Matploglib directly.

Understand the following my requirement
- mainly focusing onto distributions, categories and correlation
- histogram plot of Average Utilization of Seats for Closed Renewals, Colored by Renewal Status
- Draw a Boxplot of ACV by segment on a log scale
- Draw count plot of renewal outcome by tier
- Draw a Heatmap of the correlation matrix for the available numeric metrics, annotated and color scale fixed from -1 to 1

if you have questions understanding the above requirements, clarify with me. Do NOT assume anything on your own.
Draw each plot and generate code in different cells.

AI Prompt Engineering Specific Lessons

- Do NOT let AI assume anything.
- Do NOT let AI to generate the script | code at the first phase itself. Let AI understand your requirements thoroughly | correctly.
- Let always AI do the planning, you review and then start executing them
- Instead of asking questions to AI, let AI question you. So both of you are clarified on what you would like to design.
- Be precise in the Prompt about your requirements. If you're NOT sure, turn the AI to question and clarify with you.
- All codes | scripts generated by AI, can NOT be reviewed by humans. You must do the code review | analysis, only if code is MISSION CRITICAL | SENSITIVE functionalities.
- Any code generated by AI, the only way to prove the code is WORKING is to do the rigorous testing against all possibilities of inputs / outputs of the code.

Machine Learning - Foundations

AI -> ML -> Deep Learning -> Generative AI -> Reasoning Models -> Agentic AI

AI - Any technique that makes computers do things we would call INTELLIGENT including hand-written rules

Machine Learning - Systems that learn patterns within the data instead of being programmed rule by rule

Deep Learning - Subset of ML - Machine Learning with multiple layered neural networks - It learns its own features from raw data (images, text, audio, video, code …)

Generative AI / LLM - Subset of DL, built on the top of DL, has abilities to generate new contents based on the content trained upon.

Reasoning Models - LLMs that apply COT (Chain of Thought) and TOT (Tree of Thought) so it comes up with a plan on how to solve a complex problem

Agentic AI - With the help of Reasoning Models, and custom tools provided, it has automatically / autonomously been solving the problem with multiple iterations.

As part of Analytics, especially Predictive Analytics, ML journey starts.

Machine Learning - find the rules from examples - tries to identify intricate relationships | patterns within the data provided, which matters when the rules are TOO MANY, TOO SUBTLE or TOO CHANGABLE to write by hand.

Traditional Programming = Data + Rules -> Answers

Machine Learnings = Data + Answers -> Rules (Model)

The Learned rules are applied on new data to predict | forecast.

What ML is good at - Many Interacting Factors - Patterns Humans Miss - Consistent Scoring at Scale - Updating as Data Changes

What ML is NOT - Magic, Certainty - Or a substitute for a clear business question.

Common Vocabularies:

- Features
- Target | Label
- Training
- Model
- Prediction | Inference

ML Algorithms

- Statistical | Mathematical techniques applied on various types of data - to find best optimal way of learning relationships within the data
- Supervised Learning
- Unsupervised Learning
- Semi-Supervised Learning

+

- Manually crafted | hard-coded rules are completely avoided
    - Learns from the data points
- Works well with Structured Data - High Volume Data
- Scalability is HIGH compared to Traditional AI
- Time-to-Market
- Improvised Productivity

-

- Works well with Structured Data - Struggles with Semi-Structured | Unstructured data
- Training Models - Continuously changing data is EFFORTFUL | LABOROUS | TIME CONSUMING
- By default, ML SDKs are NOT designed, at early days, leveraging GPU hardware acceleration (They're all designed by default to use CPU)
    - Time-to-Market
    - Productivity
    - Training Cost
    - Training Time
- Feature Engineering is MANUAL

The ML Lifecycle

Seven activities define the life-cycle of ML Model | Application Development

- Problem Framing
- Data
- Features
- Label
- Model
- Evaluation
- Deployment

Let’s walk through the lifecycle

- Problem Framing
    - What decision will change?
- Data
    - Collect, Understand, Pre-process, Prepare
- Features
    - What goes into the Model? - Prepared and Cleaned
- Labels
    - Where do the right answers come from?
    - Target variable
    - Outcome
- Model
    - Choose and train an algorithm
- Evaluation
    - Is it good enough?
    - Is it measured the right way?
    - Data it has NOT seen yet, it is able to predict / forecast the target - How accurate is it then?
- Deployment
    - Trained model should be packaged in the form of a distributable
    - Inferencing - Host them, so a request can be served (new data points can be served to predict | forecast)
    - Monitoring

Types of ML (Algorithm Types)

- Supervised
- Unsupervised
- Semi-Supervised (Reinforced Learning)

Type of ML is decided by what your data contains
- Known Answers (Supervised)
- No Answers (Unsupervised)
- Rewards from Actions (Semi / Reinforcement Learning)

Supervised Learning
- Learn from Examples with Known Answers (i.e. you already have Labels)

Unsupervised Learning
- No labels
- Find structure in the data
- Clustering
- Anomaly Detection
- Dimensionality Reduction

Reinforcement Learning
- RLHF (Reinforcement Learning with Human Feedback)
- An agent takes actions, get rewards | penalties and learns a policy

In Supervised Learning
- If the target is a number, It's REGRESSION
- If the target is a category, it's CLASSIFICATION

Regression - Predicts a continuous number
- Common Metrics - Errors in Target's Units, MAE

Classification - Predicts a category
- Binary
- Multi-Class
- Common Metrics - Accuracy

Most classifiers output a PROBABILITY - then apply a threshold to turn it into a business decision

Dataset - Understanding

- Target, Features, Data Types and Business Metrics
- Before Modeling,
    - Know exactly what each one row represents
    - What the target means
    - What type of data each column is?
    - Which business metric the model should move

- Grain First
- Target
- Features by Type
    - Numerical
        - Continuous
        - Discrete
    - Categorical
        - Nominal
        - Ordinal
    - Boolean
    - Dates
    - Text
    - Identifiers
- Business Metrics vs. Model Metrics
- Logo Churn

Train - Validation - Test Data Split and Data Leakage

- Judge a model only on data it has NEVER seen
- Make sure that nothing in the inputs secretly hidden the answer
- Why Split?
    - A model scored on its own training data is marking its own homework
    - It can memorize
    - Training Dataset
        - The model learns from it
    - Validation Dataset
        - Use to make choices
    - Testing Dataset
        - Touched Once, at the end of the processing
        - Estimate real-world performance
- Stratify
    - By the target when classes are IMBALANCED
- Overfitting
    - Great at training data, worse on New Data
- Underfitting
    - Poor on both
    - Too simple
- Data Leakage
    - Information in the features that would NOT be available at prediction time
    - Indication - Results that look too good USUALLY ARE

Scikit-Learn

- It's a standard python library for classical ML on TABULAR data - One consistent toolkit for preparing data, training models and evaluating them
- SciPy-Toolkit
- It covers the whole ML classical workflow
    - Data preparation (Scaling, Encoding …)
    - Supervised Learning (Regression / Classification)
    - Unsupervised Learning (Clustering, Dimensionality Reduction, Anomaly Detection)
    - Model Selection & Tuning
    - Evaluation

Numerical Foundation - NumPy
Scientific Routines - SciPy
Data Handling - Pandas
Visualization - Matplotlib, Seaborn
Classical ML - Scikit-Learn (SK-Learn)

Every Scikit-Learn object plays ONE or MORE of FOUR Roles - Estimator, Predictor, Transformer and Meta-Estimator

Estimator - Anything that learns from the data
- Every object that learns has a FIT step
- Learning happens ONLY during FIT and ONLY from the data passed to it

Predictor - Anything that produces predictions
- After fitting, a predictor can predict outputs for new data points
- Many classifiers can produce probabilities also
- Most predictors have got a metric called SCORE that returns a default metrics for classifiers or for regressions
- Useful for a quick check

Transformer - Anything that changes the data
- A transformer transforms the data into a new column - scaling numbers, encoding categorical values, trying to minimize the number of features
- Standard practice to follow two-step patterns matters
    - Fit on Training Data (Learn the median, the categories, the scale)
    - Then transform any data (training, validation, test, live) using what was learned

Meta-Estimator - An estimator built from other estimators
- Wraps one or more estimators and behaves like an estimator itself
- A pipeline (chains steps), a grid search (tries many settings of an inner model) and ensemble (combines many models), a threshold tuner (adjusts the cut-off of an inner classifier)

Parameters and Hyper-Parameters

- Hyperparameters are the settings you choose before the training starts
- On the other hand, parameters are the values the model learns during the training

Scikit-Learn has been organized into modules by JOB.

- Data Preparation
- Composition
- Supervised Models
- Unsupervised Models
- Model Selection and Tuning
- Evaluation and Inspection

Common ML Development Workflow using Scikit-Learn

- Define X and y (Features and Target)
- Split
- Build Preparation
- Choose a Model
- Compose
- Validate and Tune
- Evaluate Once
- Save and Use

Scikit-Learn ML Demonstration - Step 1

Every SK Learn model follows the same steps

- Create
- Fit
- Predicate
- Score

Exercise #1

> in the terminal -> open a new tab

> cd c:\demonstrations

> git clone https://github.com/iomegak12/aiml_day2

> cd aiml_day2

> uv init --python=3.12

> uv sync

> uv add -r requirements.txt

> uv run main.py
(Optional Step)

> code .

> file -> auto-save

> view -> command palette -> select python interpreter -> virtual environments specific to aiml_day2 workspace

> view -> terminal

By default the virtual environment should have been deactivated if it's set at the workspace level.

> create a folder named "notebooks"

> create a file named "ml-experiment-1.ipynb" and select the kernel

> + markdown

```
# Understanding SK Learn Essentials
```

> code

```
import matplotlib.pyplot as plt
import seaborn as sns

import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.dummy import DummyClassifier, DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (accuracy_score, balanced_accuracy_score,
confusion_matrix, mean_absolute_error, r2_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree


DATA = Path("../data")
OUTPUT = Path("../output")

OUTPUT.mkdir(exist_ok=True)


subscriptions = pd.read_csv(DATA / "subscriptions.csv")
closed = subscriptions[subscriptions["renewal_status"] !=
"Open"].copy()
closed["churned"] = (closed["renewal_status"] == "Churned").astype(int)

print("Closed Renewals : ", len(closed), " | Churned: ",
closed["churned"].sum(
), " | Churn Rate : ", round(closed["churned"].mean(), 3))



FEATURES = ["avg_seat_utilization", "utilization_trend",
"avg_feature_adoption",
            "support_cases_12m", "sev1_cases_12m", "nps_score",
"term_months", "auto_renew"]

X = closed[FEATURES].astype(float)
y = closed["churned"]

print("X: ", X.shape, " | y: ", y.shape)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

print("Train :", X_train.shape, " Test: ", X_test.shape, " | Churn Rate
Train ",
      round(y_train.mean(), 3), " Test : ", round(y_test.mean(), 3))



tree = DecisionTreeClassifier(
    max_depth=3, random_state=42).fit(X_train, y_train)
predictions = tree.predict(X_test)

print("Accuracy : ", round(accuracy_score(y_test, predictions), 3),
      " | Churners Caught : ", round(recall_score(y_test, predictions),
3))

print("Confusion Matrix :", confusion_matrix(y_test, predictions))



tree_b = DecisionTreeClassifier(
    max_depth=3, class_weight="balanced", random_state=42).fit(X_train,
y_train)
predictions = tree_b.predict(X_test)

print("Accuracy : ", round(accuracy_score(y_test, predictions), 3),
      " | Churners Caught : ", round(recall_score(y_test, predictions),
3))

print("Confusion Matrix :", confusion_matrix(y_test, predictions))



importance = pd.Series(tree_b.feature_importances_,
                       index=FEATURES).sort_values(ascending=False)
print("Feature Importance by %", importance.round(3))
```

```
figure, axes = plt.subplots(figsize=(16, 7))
plot_tree(tree_b, feature_names=FEATURES,
          class_names=["Renewed", "Churned"], filled=True, ax=axes,
fontsize=8)

figure.savefig(OUTPUT / "d2_1_tree.png", dpi=110, bbox_inches="tight")

plt.show()



X_tmp, X_test2, y_tmp, y_test2 = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)

rows = []
for depth in range(1, 9):
    m = DecisionTreeClassifier(max_depth=depth,
                               class_weight="balanced",
                               random_state=42).fit(X_tr, y_tr)

    rows.append({
        "max_depth": depth,
        "train_bal_acc": balanced_accuracy_score(y_tr,
m.predict(X_tr)),
        "val_bal_acc": balanced_accuracy_score(y_val, m.predict(X_val))
    })

res = pd.DataFrame(rows).round(3)
print(res.to_string(index=False))

best = int(res.loc[res["val_bal_acc"].idxmax(), "max_depth"])

print("Best : ", best)

final = DecisionTreeClassifier(
    max_depth=best, class_weight="balanced",
random_state=42).fit(X_tmp, y_tmp)

p = final.predict(X_test2)
```

```
print(
    f"Chosen Max Depth : {best} -> Test Balanced Accuracy :
{balanced_accuracy_score(y_test2, p):3f},")
print(
    f"Accuracy : {accuracy_score(y_test2, p):3f}, Churners Caught :
{recall_score(y_test2, p):3f}")
```

Data Quality Checks - Missing Values, Duplicates, Outliers and Inconsistent Data

Audit before you fix - Measure every quality problems - Understand from where it came & record it (Lineage)

The 6 Dimensions of Data Quality

- Completeness
- Uniqueness
- Validity
- Consistency
- Accuracy
- Timeliness

Data Quality - Handling Missing Values, Imputation Strategies and When to Drop

- MCAR (Missing Completely at Random)
- Missing at Random
- Missing Not at Random

Strategies

- Drop Rows
- Drop Columns
- Mean, Median, Mode
- Constant | Unknown
- Group-wise
- Forward Fill
- Indicator Column to denote that the column value is missing

Outlier Detection

- IQR Rule
- Box Plot

> in vs code -> notebooks\2-data-quality-checks.ipynb and select the kernel

> + markdown

```
## Data Quality Checks and Audits
```

> code

```
from pathlib import Path

import pandas as pd

DATA = Path("../data")
pd.set_option("display.width", 130)
pd.set_option("display.max_columns", 30)

raw = pd.read_csv(DATA / "renewals_raw_extract.csv")



print("Shape : ", raw.shape)
print(raw.dtypes.astype(str).value_counts().to_dict())
print("Colums that are read as text that should be Numbers or Dates ? :
",
      [column for column in ["renewal_date", "auto_renew"] if
raw[column].dtype != "bool"])


raw.head()



raw.sample(5, random_state=1)
```

Missing Values

```
miss = raw.isna().sum()

miss = pd.DataFrame({
    "missing": miss,
    "pct": (miss / len(raw) * 100).round(1)
```

```
})

print(miss[miss["missing"] > 0].sort_values("missing",
ascending=False))
```

Duplicate Values

```
print("Exact Duplicate Rows :", raw.duplicated().sum())
print("Duplicate Subscription Ids : ",
raw["subscription_id"].duplicated().sum())

key = raw["subscription_id"].str.strip().str.upper()
print("Duplicated Subscription Ids :", key.duplicated().sum())

odd_ids = raw.loc[~raw["subscription_id"].str.fullmatch(r"SUB-\d{5}"),
"subscription_id"]
print("IDs not matching the Pattern ...", odd_ids.tolist())
```

Inconsistent Categories

```
for column in ["region", "segment", "tier", "product", "auto_renew"]:
    print(f"{column}:
{raw[column].value_counts(dropna=False).to_dict()}")
```

Date related Data Quality Checks

```
iso = pd.to_datetime(
    raw["renewal_date"], format="%Y-%m-%d", errors="coerce"
)

print("Parse as YYY-MM-DD : ", iso.notna().sum(),
      " Ok : ", iso.isna().sum(), "fail")
```

Valid Ranges

```
key_columns = ["seats_purchased", "discount_pct",
"annual_contract_value",
               "avg_seat_utilization", "nps_score"]

print(raw[key_columns].agg(["min", "max", "median"]).T)

rules = {
    "seats < 0 or >= 50000": (raw["seats_purchased"] < 0) |
(raw["seats_purchased"] >= 50000),
    "discount outside 0-40%": (raw["discount_pct"] < 0) |
(raw["discount_pct"] > 40),
    "ACV <= 0": raw["annual_contract_value"] <= 0,
    "utilization outside 0-1": (raw["avg_seat_utilization"] < 0) |
(raw["avg_seat_utilization"] > 1),
    "NPS outside -100 to 100": (raw["nps_score"] < -100) |
(raw["nps_score"] > 100),
}

print()
for rule, broken in rules.items():
    print(f"{rule:<24}: {broken.sum()} rows")
```

Final Data Quality Scorecard

```
issues = {
    "missing cells": int(raw.drop(columns=["renewal_acv",
"churn_reason", "csm_health", "last_csm_note"]).isna().sum().sum()),
    "duplicate rows by cleaned id": int(key.duplicated().sum()),
    "rows breaking a range rule": int(pd.concat(rules,
axis=1).any(axis=1).sum()),
    "dates not in ISO format": int(iso.isna().sum())
}

for k, v in issues.items():
    print(f"{k} - {v}")
```

Prompt for Data Quality Cleaning

can you analyze and understand the notebook notebooks\clean-the-data-3.ipynb, and i would like to implement the following data cleaning activities.

NOTE: log row counts after each step

1. standardize identifiers, categories, booleans and dates
- strip and upper-case subscription_id, and strip region, segment, tier and product
- lower case each of these four columns and map it through a dictionary of all spellings to the official value and confifrm nothing is left unmapped
- map auto_renew column eight spellings to TRUE or FALSE
- parse renewal date try %Y-%m-%d, then fill failures with %d/%m/%Y

2. Remove duplicates
- drop exact duplicates first, then drop rows that share a subscription_id, keeping the last one (the later refreshed one)

3. Fill telemetry gaps with forward fill options
The usage platform had outages, before fixing utilization we need a complete monthly series. Follow the instructions given below
A. load usage_raw_monthly.csv sorted by subscription and month - count missing active users
B. find where the gaps are - missing share by month and by region for the month of March 2026
c. Yesterdays' monthly_usage.csv happens to hold the true values so compare five ways of filling the gaps by mean absolute error: overall mean, each subscriptions own mean, forward fill, forward fill then backward fill
d. apply forward fill then backward fill within each subscription and compute each subscriptions average utilization as usage_util

4. Fix Invalid Values with Business Rules

To calculate ACV, you must follow the given formula
ACV = seats * list price * tier multiplier * (1- discount%) - if one of the three is broken and the other two are fine

compute each rows list price per seat from data\pricing_rules.json
bad seats -> rederive from ACT and discount, rounded to the nearest value
bad discount -> rederive from ACV and seats
Zero ACV -> rederive from seats and discount
utilization above 5 - > it was entered as a percentage, so divide by 100.
utilization should never be above 1 -> impossible to replace it with the usage_util
NPS outside -100 to 100 -> set to missing

5. Missing Values
create churned (0 or 1) - compare the churn rate where NPS is missing against where it's present
add an indicator nps_missing, then fill NPS with the median
fill average feature adoption with the median, industry with Unknown, employee count with the median of its segment, sales channel with the mode

fill csm health with No CSM and last csm note with No Note - these gaps are structural, not ERRORS

6. Outliers with IQR rule
write iqr_bounds return 1.5 range limits
count IQR outliers in ACV, seats, support cases and utilization along with NPS
repeat for log10(ACV)
decide for each column whether the outliers are errors or genuine. - all remaining are genuine

Save the clean file renewals_clean.csv in output folder

Do you understand my requirement enhancements? let's have a MCQs based QAs so we both are ligned in what we are trying to implement.

REFER THE NOTEBOOK FROM GitHub Repository
https://github.com/iomegak12/aiml_day2/blob/main/notebooks/clean-the-data-3.ipynb

Day 3 Agenda

- Summary of Day 1 and Day 2 Contents
- Lecture on EDA
- Demonstration on EDA
- Lecture on Feature Engineering
- Lecture on Linear Regression & Metrics
- Demonstration on Feature Engineering
- Demonstration and Practice on Linear Regression
- End-to-End Case Study (AI Assisted Development)
    - Data Engineering
    - Feature Engineering
    - Model Training | Predictions Pipeline
    - Model Serving as REST API
    - Packaging
    - Deployment
    - Monitoring & Observability

EDA - Exploratory Data Analysis | Analytics

Before you start building anything in ML, you should understand what you're working with - that step has a name - EDA

EDA - is a structured curiosity.

It's an activity that helps you to find out what's normal, what's strange, what's broken and what's interesting.

It happens at Three Levels

- Univariate
- Bivariate
- Multi-variate

Univariate means one variable at a time. We look at each column on it's own, and what does it look like?

Bivariate means two variables at a time. We look at how one column has a relationship with another column, and most importantly, how each column relates to the target variable.

Multivariate means many variables together. We look at how several things combine, overlap or even contradict each other.

One-to-One first, then pairs and then the whole group.

Univariate Analysis

- In this level, we ask commonly four different questions about every column
- Centre
- Spread Out
- Shape
- Oddities (Outliers, Gaps …)

For numbers, Histograms, Box Plots and summary statistics
For categories, Count

And there's one column we always look at first: TARGET

When the mean and median disagree that much, it's telling you something very important.

Before you compare anything, know what normal looks like for each column.

Bivariate Analysis

- This is where EDA starts answers business questions
- The common question is
    - When one thing changes, does another thing change with it?
    - Common activity
        - Pairing is every feature with the target
- Categorical Feature Against Target
- Number Against Target
- Two Numbers Against Target

Multi-variate Analysis

- Many variables at once
- This analysis answers 3 questions
    - Which features are saying the same thing?
    - Which features work together?
    - Does a pattern hold inside every group? Or overall?
- Correlation Heatmap
- Pairplot

Feature Engineering

- What a feature is? And why does the preparation of the feature matter?
- A feature is an input the model uses to make a prediction
    - utilization
    - Tier
    - Region
    - Contract value
- The target is what the model predicts - renewal value
- On business tables, better features usually beat better algorithms

Feature Types - Data Types

- Numerical
    - Continuous Numeric
    - Count
- Categorical
    - Nominal Category
    - Ordinal Category
- Boolean
- Date
- Free Text
- Identifiers

Categorical Variables

- How do you turn categories into numbers changes the way the model believes
- Encoding Techniques
    - Label Encoding
    - One-Hot Encoding
    - Ordinal Encoding
    - Target Encoding
- Label Encoding
    - You give each category an integer
    - Problem: Implies an order and distances that do NOT exist.
    - A linear model would read EMEA as twice as AMERICAS if you encode region as
        - APJC - 0
        - EMEA - 1
        - AMERICAS - 2
    - Intended for Encoding the target in Classification, NOT features

- One-Hot Encoding
    - It accomplishes this by creating a new binary column for every unique category values in your dataset
    - For any given row, a 1 is placed in the column corresponding to it's category, while all other columns receive a 0
    - No False Order

        - Cost - Many columns for high-cardinality features
        - Unseen Categories - Production data may contain a value training never saw - A robust encoder outputs all zeros rather than failing

- Ordinal Encoding
    - Integers that follow a real order
    - Tier - Essentials (0), Advantage (1), Premier (2)
    - Assumes equal steps between levels
    - Always state the order explicitly
    - The default alphabetical order is USUALLY WRONG

- Target Encoding
    - Replaces each category by the average target for the category
        - For example, each industry's average renewal change
    - Compact - Ideal for features with many categories
        - Such as Country, Product Code, Account
    - Two risks
        - Noisy Small Groups -> Smoothing
            - Blend the group's average with the overall average
        - Leakage -> Cross Fitting
            - Fit on the training data only

- Other Encoding (NOT COMMONLY USED)
    - Frequency Encoding
    - Grouping Rare Categories
        - Introduce a new category "Others", "Unknown", "Miscellaneous" before one-hot encoding
    - Hashing
        - Map many categories into a fixed number of columns

Numerical Features

- Scaling and Transforming
- Scaling - makes features comparable
- Transforming - changes their shape
- 5 Different Methods

Method	What does it do?	When is it used?	Watch out?
Standardization (Z-Score)	Subtract Mean, Divide by Standard Deviation	Default for Linear Models - Regularization - Logistic Regression - PCA	Not bounded, outliers stay extreme
Normalization (Min-Max)	Rescale 0-1	A bounded range is required	

Robust Scaling	Subtract Median, Divide by IQR	Data with Outliers	
Log Transform	Replace it's value with logarithm	Right Skewed - Positive Values (Money, Counts, Sizes)	
Power Transform	Families that make data more bell shaped	Strong Skew of Unknown Form	

Feature Selection

- Fewer, Better features give simpler, more stable and importantly more explainable models
- Wrapper (Recursive Feature Elimination)
- Embedded (Lasso based algorithms - Coefficients based - Tree importances based)

Pipelines & Column Transformers

- Bundling preparation and model into one object makes the workflow - reproducible, leakage free and easy to deploy
- Column Transformers - send each group of columns into it's own data preparation chains
    - For example, numbers -> fill gaps with the training median -> standardization
    - Categories of type nominal -. Fill gaps with the most common value -> one-hot encoding
- One artifact to save and deploy

Demonstration for Feature Engineering

> in the new terminal window,

> cd c:\demonstrations

> git clone https://github.com/iomegak12/aiml-day3

> cd aiml-day3

> uv init --python=3.12

> uv sync

> uv add -r requirements.txt

> code .

> in vs code -> view -> command palette -> select python interpreter -> virtual environments -> choose workspace specific virtual environment

> view -> terminal

You should be able to see by default virtual environment is activated when you open a terminal window, which indicates that workspace specific virtual environment has been successfully activated

> create a folder named “notebooks”

> create a file named “1-feature-engineering.ipynb” and select the kernel (make sure that you select the same virtual environment as your workspace)

> + markdown

Feature Engineering

> + code

```
from pathlib import Path

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import VarianceThreshold,
mutual_info_regression
from sklearn.impute import SimpleImputer
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import (
    LabelEncoder, minmax_scale, OneHotEncoder, OrdinalEncoder,
RobustScaler, StandardScaler, TargetEncoder)
```

>

```
DATA = Path("../data")
pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 130)


df = pd.read_csv(DATA / "renewals_day3.csv",
                 parse_dates=["renewal_date", "customer_since"])

ren = df[df["renewal_status"] == "Renewed"].copy()
ren["change_pct"] = ren["renewal_acv"] / ren["annual_contract_value"] -
1

print("Renewed Subscriptions :", len(ren))
print(ren["change_pct"].describe().round(2).to_dict())



train, test = train_test_split(ren, test_size=0.2, random_state=42)

print("Train : ", train.shape, " | Test: ", test.shape)
print("Missing Values in Train :", train[[
      "industry", "employee_count", "sales_channel",
"avg_feature_adoption", "nps_score"]].isna().sum().to_dict())
```

> + markdown

Encoding Categorical Columns

> + code

```
# Label Encoding: each region gets an integer code, assigned in
alphabetical order.
```

```
# LabelEncoder is meant for target labels; using it on a feature
creates a fake numeric order.
le = LabelEncoder()
le.fit(train["region"])
print(dict(zip(le.classes_, le.transform(le.classes_).tolist())))


# One-Hot Encoding: one 0/1 column per category, so no fake order
between categories.
# handle_unknown="ignore" turns an unseen category (LATAM) into all
zeros instead of raising an error.
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
ohe.fit(train[["segment", "region"]])
print("New Columns :", ohe.get_feature_names_out().tolist())

new_row = pd.DataFrame({"segment": ["SMB"], "region": ["LATAM"]})
print(pd.DataFrame(ohe.transform(new_row),
columns=ohe.get_feature_names_out()))


# Ordinal Encoding: we pass the business order explicitly, so
Essentials=0 < Advantage=1 < Premier=2.
# Without `categories=`, sklearn would sort alphabetically (Advantage,
Essentials, Premier), which is wrong here.
oe = OrdinalEncoder(categories=[["Essentials", "Advantage",
"Premier"]])
oe.fit(train[["tier"]])

tiers = pd.DataFrame({"tier": ["Essentials", "Advantage", "Premier"]})
print(tiers.assign(tier_code=oe.transform(tiers)))



# Target Encoding: each industry becomes a smoothed mean of change_pct
(rare categories are pulled toward the overall mean).
# Missing industries are filled with "Unknown" first so they get their
own category.
industry = train[["industry"]].fillna("Unknown")

te = TargetEncoder(target_type="continuous", random_state=42)
te.fit(industry, train["change_pct"])

comparison = pd.DataFrame({"target_encoding": te.encodings_[0]},
index=te.categories_[0])
```

```
comparison["raw_mean"] =
train["change_pct"].groupby(industry["industry"]).mean()
comparison["rows"] = industry["industry"].value_counts()

print(comparison.round(3))
print("Overall Mean :", round(train["change_pct"].mean(), 3))
```

> code

```
# Scaling: fit each scaler on the TRAIN values only, then transform
train and test with those same train statistics.
# Standard -> mean 0 / std 1, MinMax -> train range [0, 1], Robust ->
median 0 / IQR 1 (less affected by outliers).
from sklearn.preprocessing import MinMaxScaler

num_cols = ["annual_contract_value", "avg_seat_utilization"]
scalers = {"Standard": StandardScaler(), "MinMax": MinMaxScaler(),
"Robust": RobustScaler()}

rows = []
for name, scaler in scalers.items():
    scaled_train = pd.DataFrame(scaler.fit_transform(train[num_cols]),
columns=num_cols)
    scaled_test = pd.DataFrame(scaler.transform(test[num_cols]),
columns=num_cols)
    rows.append({
        "scaler": name,
        "train_acv_mean": scaled_train["annual_contract_value"].mean(),
        "train_acv_std":
scaled_train["annual_contract_value"].std(ddof=0),  # ddof=0 matches
how StandardScaler computes std
        "train_acv_median":
scaled_train["annual_contract_value"].median(),
        "test_acv_min": scaled_test["annual_contract_value"].min(),
        "test_acv_max": scaled_test["annual_contract_value"].max(),
        "train_util_min": scaled_train["avg_seat_utilization"].min(),
        "train_util_max": scaled_train["avg_seat_utilization"].max(),
    })

print(pd.DataFrame(rows).set_index("scaler").round(3))
```

>

```
# How squashed is ACV under MinMax? A few very large contracts set the
max, so most rows crowd near 0.
# We count the share of training rows that land in the bottom 10% of
the [0, 1] range.
acv_minmax =
MinMaxScaler().fit_transform(train[["annual_contract_value"]])
print(f"Train ACV Rows Below 0.1 after MinMax : {(acv_minmax <
0.1).mean():.1%}")
```

> + markdown

Feature Engineering - Adding More Features

```
# add_features(): builds five row-wise features, i.e. each row uses
only its own values.
# Because nothing is learned from other rows, it is safe to apply to
train and test separately (no leakage).
def add_features(data):
    data = data.copy()
    data["tenure_years"] = (data["renewal_date"] -
data["customer_since"]).dt.days / 365.25
    data["log_acv"] = np.log(data["annual_contract_value"])
    data["acv_per_seat"] = data["annual_contract_value"] /
data["seats_purchased"]
    data["support_per_100_seats"] = data["support_cases_12m"] /
data["seats_purchased"] * 100
    data["had_sev1"] = (data["sev1_cases_12m"] >= 1).astype(int)
    return data


train = add_features(train)
test = add_features(test)
print("Train :", train.shape, " | Test :", test.shape)


# Describe the five new features on train: check their ranges and skew
(mean vs median).
# For had_sev1 (0/1), the mean is the share of accounts with at least
one Sev-1 case.
```

```
new_features = ["tenure_years", "log_acv", "acv_per_seat",
"support_per_100_seats", "had_sev1"]
print(train[new_features].describe().T.round(2))
```

```
# Correlate each new feature with change_pct using Pearson, on the
TRAIN set only (test stays untouched).
# Raw ACV is included so we can compare it against log ACV; rows are
sorted by strength, ignoring the sign.
corr = train[new_features +
["annual_contract_value"]].corrwith(train["change_pct"])
print(corr.sort_values(key=abs, ascending=False).round(3))
```

> + markdown

Feature Selection

>

```
# Variance Threshold: one-hot encode industry (gaps -> "UNKNOWN"), then
drop dummies with variance below 0.02.
# A 0/1 dummy has variance p*(1-p), so 0.02 removes categories present
in fewer than ~2% of rows.
industry_ohe = OneHotEncoder(sparse_output=False)
industry_dummies = pd.DataFrame(
    industry_ohe.fit_transform(train[["industry"]].fillna("UNKNOWN")),
    columns=industry_ohe.get_feature_names_out())

vt = VarianceThreshold(threshold=0.02)
vt.fit(industry_dummies)

print(pd.DataFrame({"variance": vt.variances_, "keep":
vt.get_support()}, index=industry_dummies.columns).round(4))
print("Dropped :",
industry_dummies.columns[~vt.get_support()].tolist())
```

>

```
# High correlation: find pairs of numeric features with |r| > 0.85 on
train; one of each pair is usually redundant.
# renewal_acv is left out because change_pct is calculated from it
(target leakage).
num_features = ["employee_count", "seats_purchased", "discount_pct",
"annual_contract_value", "term_months",
                "avg_seat_utilization", "utilization_trend",
"avg_feature_adoption", "support_cases_12m",
                "sev1_cases_12m", "nps_score"] + new_features

corr_matrix = train[num_features].corr().abs()
pairs = []
for i, a in enumerate(num_features):
    for b in num_features[i + 1:]:
        if corr_matrix.loc[a, b] > 0.85:
            pairs.append((a, b, round(corr_matrix.loc[a, b], 3)))

print(pd.DataFrame(pairs, columns=["feature_1", "feature_2",
"abs_corr"]))
```

```
>
```

```
# Mutual Information: scores how much knowing a feature tells us about
change_pct, including non-linear links (0 = nothing).
# MI cannot handle NaNs, so gaps are filled with the train median
first.
X_mi =
SimpleImputer(strategy="median").fit_transform(train[num_features])
mi = mutual_info_regression(X_mi, train["change_pct"], random_state=42)
print(pd.Series(mi,
index=num_features).sort_values(ascending=False).round(3))


# Leakage Check: a feature is only usable if it is known BEFORE the
renewal is priced.
# open_renewals.csv holds renewals not priced yet, so any column
missing there only exists after the outcome.
open_cols = pd.read_csv(DATA / "open_renewals.csv", nrows=0).columns
print("Not Available Before Pricing :", [c for c in df.columns if c not
in open_cols])
print("utilization_trend Available Before Pricing :",
"utilization_trend" in open_cols)
```

```
# Permutation Importance: fit a model on train, then shuffle one
feature at a time on TEST and measure how much R2 drops.
# A big drop = the model really relies on that feature; ~0 = the
feature can be removed without loss.
from sklearn.inspection import permutation_importance

model = make_pipeline(SimpleImputer(strategy="median"),
StandardScaler(), LinearRegression())
model.fit(train[num_features], train["change_pct"])
print("Test R2 :", round(r2_score(test["change_pct"],
model.predict(test[num_features])), 3))

perm = permutation_importance(model, test[num_features],
test["change_pct"], scoring="r2", n_repeats=10, random_state=42)
print(pd.Series(perm.importances_mean,
index=num_features).sort_values(ascending=False).round(3))
```

```
# Lasso (L1): a linear model that pushes weak or redundant feature
coefficients to exactly 0, i.e. it selects features.
# LassoCV picks the penalty strength (alpha) with 5-fold CV on train;
features scaled first so coefficients are comparable.
from sklearn.linear_model import LassoCV

lasso = make_pipeline(SimpleImputer(strategy="median"),
StandardScaler(), LassoCV(cv=5, random_state=42))
lasso.fit(train[num_features], train["change_pct"])

coefs = pd.Series(lasso[-1].coef_, index=num_features)
print("Chosen alpha :", round(lasso[-1].alpha_, 5))
print(coefs.sort_values(key=abs, ascending=False).round(4))
print("Dropped by Lasso (coef = 0) :", coefs[coefs ==
0].index.tolist())
```

> + markdown

Pipeline with Column Transformer

>

```
# preprocess: one ColumnTransformer with three branches, each applying
its own steps to its own columns (others are dropped).
# set_output("pandas") keeps column names; this needs
sparse_output=False on the OneHotEncoder.
nominal_cols = ["segment", "region", "product", "sales_channel",
"industry"]
ordinal_cols = ["tier"]

preprocess = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
StandardScaler()), num_features),
    ("nom", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore",
sparse_output=False)), nominal_cols),
    ("ord", OrdinalEncoder(categories=[["Essentials", "Advantage",
"Premier"]]), ordinal_cols),
]).set_output(transform="pandas")
```

>

```
# Full pipeline: preprocess -> LinearRegression, fitted on TRAIN only.
# Every imputer, scaler and encoder inside learns from train, then the
same fitted steps are reused on test.
pipe = Pipeline([("preprocess", preprocess), ("model",
LinearRegression())])
pipe.fit(train, train["change_pct"])


# Inspect: transformed test shape, first six columns, the medians the
numeric imputer learnt (from train), and MAE / R2.
# Train vs test metrics side by side show whether the model overfits.
test_transformed = pipe["preprocess"].transform(test)
print("Transformed Test Shape :", test_transformed.shape)
print(test_transformed.iloc[:, :6].head().round(3))

num_imputer = pipe["preprocess"].named_transformers_["num"][0]
print("Medians Learnt by the Numeric Imputer :")
print(pd.Series(num_imputer.statistics_, index=num_features).round(3))

for name, data in [("Train", train), ("Test", test)]:
    pred = pipe.predict(data)
```

```
    print(f"{name} MAE : {mean_absolute_error(data['change_pct'],
pred):.4f} | R2 : {r2_score(data['change_pct'], pred):.3f}")
```

Prompts used in Feature Engineering | Feature Selection

can you analyze and understand notebooks\1-feature-preparation.ipynb notebook, where I shall be demonstrating various types of feature engineering and pipeline capabilities.

let me know what you have understood in a line or two.

do NOT generate anything | code | scripts | commands ...

I shall be focusing onto Encoding categories (espcially categorical columns)
Your task:

- Label Encoding - fit a label encoder on region and print the codes - explain me in a line what would a linear model conclude from them?
- one hot encoding - fit one hot encoder on segment, region - ignore the unknowns - print the new column names, then transform a new row with the region LATAM
- ordinal encoding - encoder tier with the explicit order (Essentials < Advantage < Premier)
- target encoding - fit a target encoder on industry (fill gaps with unknown first it they're missing) against change percentage - compare it's encodings with the raw mean per industry and the number of rows that you have in the dataset

do you get my requirements related to categorical encodings? if no, clarify with me

NOTES:
- do NOT assume by yourselves
- do NOT generate too much of complicated codes
- it should be easy to understand | troubleshoot | debug
- add explanatory documentation comments just above script (1-2 lines)

Like categorical columns, now i would like to focus onto numerical columns ...

Your task is to implement the following requirement
- we have two columns importantly named annucal contract value and average seat utilization
- fit standard scaler, min max scaler and robust scaler on the training values of these two columns
- for each, print the training ACV mean, standard deviation, median after scaling, the test ACV range, and the training utilization range
- then check how squashed ACV is under min max scaling

describe the cell with appropriate comments

if you have any questions, clarify with me.

do I have to put any efforts to understand the meaning of this output from the cell [11]

Train ACV Rows Below 0.1 after MinMax : 75.8%

as part of my feature engineering exercise, i would like to now add the following features to be implemented

- write add_features function that adds five rows_wise features, apply them to training and test dataset
- describe them
- importantly correlate them with change percentage on the training set
- features
a. tenure in years = years from customer since to renewal date
b. log acv = natural log of ACV
c. acv per seat = price actually paid per seat
d. support per 100 seats = support cases per 100 seats
e. had severity 1 = 1 if there was atleast one Sev-1 case

let me know if there are any clarifications needed

can you explain the output of cell [14] as i am little bit need of explanations

<pasted_content id="f774">
log_acv -0.099
annual_contract_value -0.085
support_per_100_seats 0.074
acv_per_seat -0.069
tenure_years 0.040
had_sev1 0.022
dtype: float64
</pasted_content id="f774">

brief me in simple words, in 1-2 lines

from the cell [10], i got the output as below

<pasted_content id="f774">
train_acv_mean train_acv_std train_acv_median test_acv_min test_acv_max train_util_min train_util_max
scaler
Standard 0.000 1.000 -0.426 -0.571 3.455 -2.851 2.743
MinMax 0.087 0.152 0.022 0.000 0.612 0.000 1.000
Robust 0.705 1.656 0.000 -0.241 6.424 -2.180 2.073
</pasted_content id="f774">

out of which, which scaler do you recommend for columns annual contract value and average seat utilization? just brief me in 1-2 lines

as per of my feature engineering experiments, now, i would like to focus onto feature selection with the following requirements
- apply one hot encoding to industry column (all gaps can be considered as UNKNOWN) and apply variance thres hold to see which dummy values can be dropped?

- find pairs of numerical features with absolute correlation above 85%

- rank numerical features by mutual information with change percentage

do you recommend me any additional feature selection techniques apart from the above in the given dataset?

explain me why in brief in 1-2 lines.

if you have any questions, clarify with me

can you implement these recommended techniques as part of the notebook

Now, i believe i have completed feature engineering and feature selection together from the dataset ...

I want you now to focus onto building a pipeline that puts all of them together using Column Transformer

Your task is to satisfy the following requirements

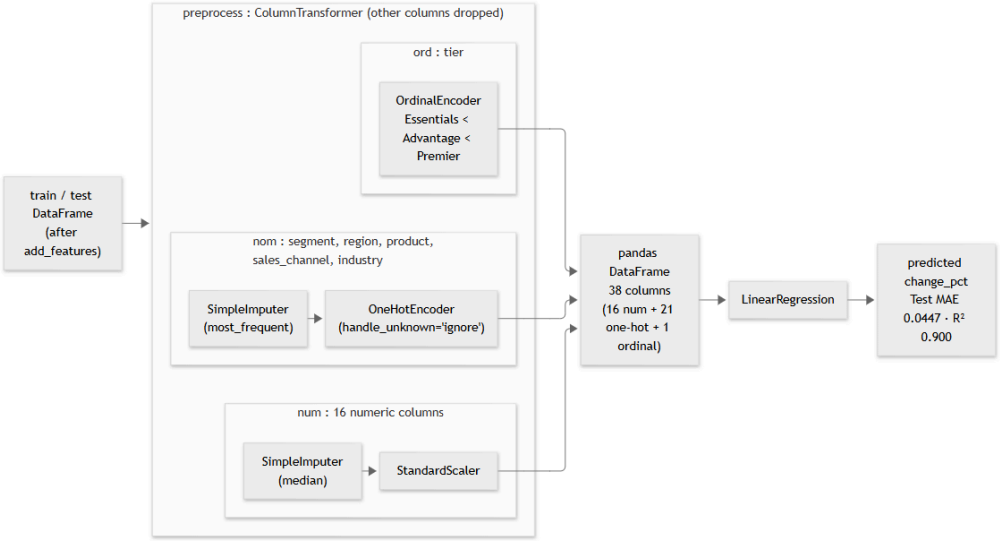
- Build a column transformer named pre process with three different branches
a. numeric columns (median imputation, then standard scaling)
b. nominal columns (segment, region, product, sales channel and industry) - most frequent imputation and then one hot encoding that ignores unknown categories
c. ordinal (tier) : ordinal encoding with the business order
- set it to output pandas data frames, chain it with Linear Regression in a pipeline - fit on train to predict change percentage
- print the transformed test shape, the first six columns, the medians that imputation has learnt and the MAE and R-square errors

any inputs needed from my end still? do you get my points? ask me questions if you need clarifications.

NOTE: do NOT assume anything on your own.

can you describe the pipeline in mermaid diagram format

Pipeline Diagram



Supervised Learning

- Regression (Continuous Numerical)
- Classification (Categorical)

Regression

- What does Regression do?
    - Regression predicts a number by learning how the target changes as and when the features change
    - Predicts a continuous value
    - It answers two questions at once
        - Prediction
            - What will the renewal be worth?
        - Explanation
            - What moves the renewal value and how much?

- Every regression model has three ingredients
    - A Model form (line, curve, tree)
    - A Cost function (how wrong is it?)
    - A Learning Method (how it reduces the cost)

- Always compare with Baselines

Simple Linear Regression

- Fit the straight line that makes the squared vertical gaps between points and line as small as possible
- Residual - actual - predicted - the vertical gap from point to line
- Line of Best Fit = the line with the smallest sum of squared residuals

How does the line is found in the Linear Regression?

- Cost
    - MSE (Mean Squared Errors)
- OLS
    - Ordinary Least Squares
- Gradient Descent
    - Start with any slope and intercept, measure which direction is downhill on the cost

Multiple Linear Regression

- Each coefficient (feature to the target) is the effect of one feature holding the others fixed - and it only makes sense if the models' form matches the business

When it is NOT the right tool?

- Use it when
    - Relationships are roughly linear
    - Data is modest in size

        - Explanation matters as much as prediction

- Consider alternatives when
    - Relationships are high NON-LINEAR with many interactions
        - Tree based
        - Gradient Boosting
        - Ensembling Techniques
    - The target is a category
    - The data is SEMi / UNSTRUCTURED

Regression Metrics and Evaluation

- Core Metrics
    - 6 Core Metrics
    - MAE (Mean Absolute Error)
    - MSE (Mean Squared Error)
    - RMSE (Root Mean Squared Error)
    - R2 (Coefficient of Determination)
    - Adjusted R2 (R2 penalized for the number of features)
    - MAPE (Mean Absolute Percentage Error)

Order	Actual (Minutes)	Predicted (Minutes)	Miss
A	30	28	2 Early
B	25	27	2 Late
C	40	38	2 Early
D	20	22	2 Late
E	35	45	10 Late

MAE: “On Average, how far off are we?”

Ignore whether each miss late | early, and average the size of the misses (2+2+2+2+10 / 5 = 3.6 minutes)

MSE - Mean Squared Error - Punish big misses harder

Square each miss before averaging - (4+4+4+4+100)/5 = 23.2 square minutes

The now 10 minute miss now contributes 100 of the 116 total

This value is NOT generally meant for business, it’s for internal maths purposes.

RMSE: MSE translated back into minutes

Take the square root of MSE = Square root of 23.2 = 4.8 minutes

A few disasters dominate it. Compare it with MAE - Here MAE is 3.6 and RMSE is 4.8 is noticeably above MAE, which tells you …

Mostly fine, but with occasional big misses

IF the two were nearly equal, the errors would be evenly sized.

R2 (Coefficient of Determination) - How much better than a lazy guess?

- The lazy guess is to always to predict the average - 30 Minutes Every Order
- R2 measures how much of that lazy guess error your model removes
- 1.0 - means perfect
- 0 - means no better than lazy guess
- Below 0 - means worse than lazy guess

Adjusted R2: R2 that can't be gamed by adding features

- It's mainly used in Classical statistics, and in Modern ML, it's known as cross validation

MAPE: (Mean Absolute Percentage Error) - How far off, as a percentage?

- Divide each miss by the actual value, then average

Simple Linear Regression - Demonstration for Practice

> in vs code -> in the notebooks folder -> create a file named "3-linear-regression.ipynb" and select the kernel

> + markdown

Learning Linear Regression Professionally

```
from pathlib import Path

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer,
TransformedTargetRegressor
from sklearn.feature_selection import VarianceThreshold,
mutual_info_regression
```

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, ElasticNet, Lasso,
Ridge
from sklearn.metrics import mean_absolute_error, r2_score,
mean_absolute_percentage_error, mean_squared_error
from sklearn.model_selection import train_test_split, cross_validate,
GridSearchCV
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import (
    LabelEncoder, minmax_scale, OneHotEncoder, OrdinalEncoder,
RobustScaler, StandardScaler, TargetEncoder)

import warnings


>

warnings.filterwarnings("ignore", category=UserWarning)

DATA = Path("../data")
OUTPUT = Path("../output")
OUTPUT.mkdir(exist_ok=True)

pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 130)



df = pd.read_csv(DATA / "renewals_day3.csv",
                 parse_dates=["renewal_date", "customer_since"])

ren = df[df["renewal_status"] == "Renewed"].copy()
ren["change_pct"] = ren["renewal_acv"] / ren["annual_contract_value"] -
1


def add_features(data):
    data = data.copy()
    data["tenure_years"] = (data["renewal_date"] -
                            data["customer_since"]).dt.days / 365.25
    data["log_acv"] = np.log(data["annual_contract_value"])
    data["acv_per_seat"] = data["annual_contract_value"] / \
        data["seats_purchased"]
```

```
    return data


ren = add_features(ren)
train, test = train_test_split(ren, test_size=0.2, random_state=42)


NUM_RAW = ["annual_contract_value", "seats_purchased", "discount_pct",
"avg_seat_utilization", "utilization_trend",
           "avg_feature_adoption", "support_cases_12m", "nps_score",
"employee_count", "term_months"]
NUM = ["log_acv", "discount_pct", "avg_seat_utilization",
"utilization_trend",
       "avg_feature_adoption", "support_cases_12m", "nps_score",
"term_months", "tenure_years", "acv_per_seat"]
CATEGORICAL = ["segment", "region", "product", "sales_channel",
"industry"]
ORDINAL = ["tier"]


def make_preprocess(numeric):
    return ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"),
StandardScaler()), numeric),
        ("nom", make_pipeline(SimpleImputer(strategy="most_frequent"),
OneHotEncoder(
            handle_unknown="ignore", sparse_output=False)),
CATEGORICAL),
        ("ord", OrdinalEncoder(categories=[
         ["Essentials", "Advantage", "Premier"]]), ORDINAL)
    ])


def evaluate(name, y_true, y_pred):
    return {
        "model": name,
        "MAE $": mean_absolute_error(y_true, y_pred),
        "RMSE $": mean_squared_error(y_true, y_pred) ** 0.5,
        "R2 $": r2_score(y_true, y_pred),
        "MAPE %": mean_absolute_percentage_error(y_true, y_pred) * 100
    }
```

```
def show(rows):
    t = pd.DataFrame(rows)

    print(t.to_string(index=False))



y_test = test["renewal_acv"]



rows = [
    evaluate("Renews at Todays' ACV:", y_test,
             test["annual_contract_value"]),
    evaluate("Todays ACV + 3% uplift : ", y_test,
             test["annual_contract_value"] * 1.03)
]

show(rows)



X_COL, Y_COL = "utilization_trend", "change_pct"

simple = LinearRegression().fit(train[[X_COL]], train[Y_COL])
b0, b1 = simple.intercept_, simple.coef_[0]

print(f"Equation : change_pct = {b0:.4f} + {b1:.4f} *
utilization_trend")
print(f"           (intercept = expected change when trend is flat,
i.e. trend = 0 -> {b0:+.1%})\n")

rows = []
for name, part in [("train", train), ("test", test)]:
    pred = simple.predict(part[[X_COL]])
    rows.append({
        "split": name,
        "n": len(part),
        "R2": r2_score(part[Y_COL], pred),
        "MAE (pp)": mean_absolute_error(part[Y_COL], pred) * 100,
        "RMSE (pp)": mean_squared_error(part[Y_COL], pred) ** 0.5 *
100,
```

```
    })
show(rows)

r2_train, r2_test = rows[0]["R2"], rows[1]["R2"]
print(f"\nR2 test = {r2_test:.3f} -> utilization_trend alone explains
~{r2_test:.0%} of the variance "
      f"in change_pct on unseen accounts (train {r2_train:.3f}, gap
{r2_train - r2_test:+.3f}).")

step = 0.10
print(f"\nWhat a +{step:.2f} change in utilization_trend means:")
print(f"  slope * {step:.2f} = {b1:.4f} * {step:.2f} = {b1 *
step:+.4f}"
      f"  ->  renewal change moves by {b1 * step * 100:+.1f} percentage
points")
for t in [-0.10, 0.00, 0.10]:
    print(f"  trend = {t:+.2f}  ->  predicted change_pct = {b0 + b1 *
t:+.1%}")
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 6))

ax.scatter(train[X_COL], train[Y_COL], s=22, alpha=0.45,
color="#2a78d6",
           edgecolor="white", linewidth=0.5, label=f"train
(n={len(train)})")
ax.scatter(test[X_COL], test[Y_COL], s=30, alpha=0.9, color="#eb6834",
           edgecolor="white", linewidth=0.5, label=f"test
(n={len(test)})", zorder=3)

# Residuals on the test set: least squares picks the line that
minimises the sum of these squared vertical gaps
test_pred = simple.predict(test[[X_COL]])
ax.vlines(test[X_COL], test_pred, test[Y_COL], color="#9a9a94",
linewidth=0.8, alpha=0.6,
          label="residual (actual - predicted)", zorder=2)

xs = np.linspace(ren[X_COL].min(), ren[X_COL].max(), 100)
```

```
ax.plot(xs, b0 + b1 * xs, color="#1f1f1d", linewidth=2,
        label=f"fit: y = {b0:.3f} + {b1:.3f}x", zorder=4)

# Slope triangle: +0.10 in trend -> +b1*0.10 in change_pct
x0 = 0.25  # placed where points are sparse so the triangle is readable
y0 = b0 + b1 * x0
ax.plot([x0, x0 + step], [y0, y0], color="#1f1f1d", linewidth=1.6,
linestyle="--", zorder=5)
ax.plot([x0 + step, x0 + step], [y0, y0 + b1 * step], color="#1f1f1d",
linewidth=1.6, linestyle="--", zorder=5)
ax.annotate(f"+{step:.2f} trend", (x0 + step / 2, y0), xytext=(0, -14),
textcoords="offset points",
            ha="center", fontsize=9, color="#1f1f1d")
ax.annotate(f"{b1 * step * 100:+.1f} pp change", (x0 + step, y0 + b1 *
step / 2), xytext=(6, 0),
            textcoords="offset points", va="center", fontsize=9,
color="#1f1f1d")

ax.axhline(0, color="#cfcfca", linewidth=0.8, zorder=0)
ax.axvline(0, color="#cfcfca", linewidth=0.8, zorder=0)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _:
f"{v:.0%}"))
ax.set_xlabel("utilization_trend")
ax.set_ylabel("change_pct  (renewal ACV / current ACV - 1)")
ax.set_title(f"Renewal price change vs utilization trend  |  R2 train
{r2_train:.3f}, test {r2_test:.3f}",
             loc="left")
ax.spines[["top", "right"]].set_visible(False)
ax.grid(alpha=0.25)
ax.legend(loc="upper left", frameon=False)
plt.tight_layout()
plt.show()




def predict_renewal(model, accounts):
    """Predict change_pct and renewal ACV for new account(s).

    accounts: a dict (one account) or a DataFrame; needs
utilization_trend and annual_contract_value.
    """
```

```
    new = pd.DataFrame([accounts]) if isinstance(accounts, dict) else
accounts.copy()

    out = pd.DataFrame({
        X_COL: new[X_COL],
        "current_acv": new["annual_contract_value"],
        "pred_change_pct": model.predict(new[[X_COL]]),
    }, index=new.index)
    out["pred_renewal_acv"] = out["current_acv"] * (1 +
out["pred_change_pct"])

    if "renewal_acv" in new:
        out["actual_change_pct"] = new["renewal_acv"] /
new["annual_contract_value"] - 1
        out["actual_renewal_acv"] = new["renewal_acv"]
        out["error $"] = out["actual_renewal_acv"] -
out["pred_renewal_acv"]

    lo, hi = train[X_COL].min(), train[X_COL].max()
    outside = ~out[X_COL].between(lo, hi)
    if outside.any():
        print(f"Warning: {outside.sum()} row(s) have {X_COL} outside
the training range "
              f"[{lo:+.3f}, {hi:+.3f}] - the line is extrapolating
there.\n")

    pct = "{:+.1%}".format
    usd = "${:,.0f}".format
    print(out.to_string(formatters={
        X_COL: "{:+.3f}".format, "current_acv": usd, "pred_change_pct":
pct, "pred_renewal_acv": usd,
        "actual_change_pct": pct, "actual_renewal_acv": usd,
        "error $": lambda v: f"{'+' if v >= 0 else '-'}${abs(v):,.0f}",
    }))
    return out


def show_metrics(model, splits=None):
    """Score the model on change_pct and on renewal ACV in $ (with the
today's-ACV baseline for comparison)."""
    splits = splits or {"train": train, "test": test}

    change_rows, dollar_rows = [], []
```

```
    for name, part in splits.items():
        pred_change = model.predict(part[[X_COL]])
        change_rows.append({
            "split": name,
            "n": len(part),
            "R2": r2_score(part[Y_COL], pred_change),
            "MAE (pp)": mean_absolute_error(part[Y_COL], pred_change) *
100,
            "RMSE (pp)": mean_squared_error(part[Y_COL], pred_change)
** 0.5 * 100,
        })
        pred_acv = part["annual_contract_value"] * (1 + pred_change)
        dollar_rows.append(evaluate(f"trend model ({name})",
part["renewal_acv"], pred_acv))

    last = list(splits.values())[-1]
    dollar_rows.append(evaluate("baseline: today's ACV",
last["renewal_acv"], last["annual_contract_value"]))

    print("On change_pct (what the model predicts):")
    show(change_rows)
    print("\nOn renewal ACV in $ (baseline scored on the last split):")
    show(dollar_rows)
```

```
# 1) A brand-new account: only the inputs are known
new_account = {"annual_contract_value": 120_000, "utilization_trend":
0.12}
print("New account:")
predict_renewal(simple, new_account)

# 2) A few held-out test accounts: actuals are known, so we can see the
error per row
print("\nHeld-out test accounts:")
predict_renewal(simple, test.sample(5, random_state=7))

# 3) Overall metrics
print()
show_metrics(simple)
```

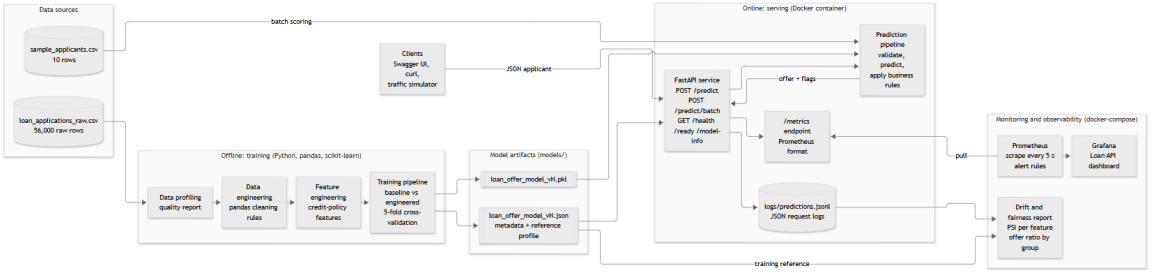
Day 4 Agenda

- Quick summary of Day 1 - 3 Contents
- Understand the Business Use Case for End-to-End Case Study
- Demonstration & Practices - End-to-End Case Study using AI Assisted Development
- Lecture on Logistic Regression & Related Metrics
- Demonstration & Practices on Logistic Regression Models

End-to-End Case Study on ML Application Development Lifecycle

Scenario:

- Northwind Finance
- Loan Application Processing
- Maximum Loan Amount it can offer - through an API, and to know at all times whether that service is HEALTHY, ACCURATE and FAIR
- Model: Linear Regression
- Engineered on Credit Policy Features
- API is served by FastAPI
- Docker Container
- Monitoring & Observability through Prometheus & Grafana



Mermaid Contents for Architecture Diagram (https://mermaid.live)

```
flowchart LR
    subgraph SRC["Data sources"]
        RAW[("loan_applications_raw.csv<br/>56,000 raw rows")]
        SAMPLE[("sample_applicants.csv<br/>10 rows")]
    end

    subgraph OFFLINE["Offline: training (Python, pandas, scikit-learn)"]
        PROFILE["Data profiling<br/>quality report"]
        CLEAN["Data engineering<br/>pandas cleaning rules"]
        FE["Feature engineering<br/>credit-policy features"]
        TRAIN["Training pipeline<br/>baseline vs engineered<br/>5-fold cross-validation"]
    end

    subgraph REG["Model artifacts (models/)"]
        PKL[["loan_offer_model_vN.pkl"]]
        META[["loan_offer_model_vN.json<br/>metadata + reference profile"]]
    end

    subgraph SERVE["Online: serving (Docker container)"]
        API["FastAPI service<br/>POST /predict<br/>POST /predict/batch<br/>GET /health
/ready /model-info"]
        PRED["Prediction pipeline<br/>validate, predict,<br/>apply business rules"]
        MET["/metrics endpoint<br/>Prometheus format"]
        LOG[("logs/predictions.jsonl<br/>JSON request logs")]
    end

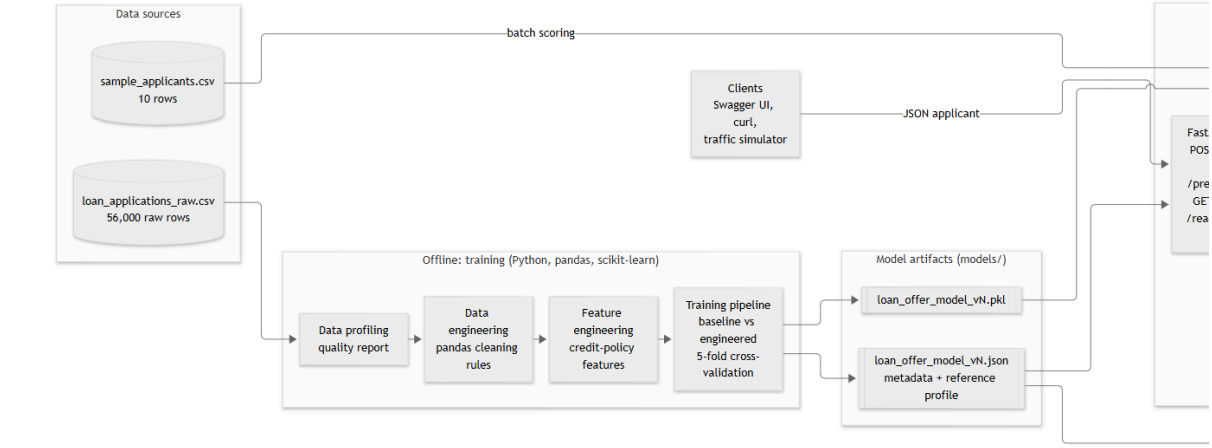
    subgraph OBS["Monitoring and observability (docker-compose)"]
        PROM["Prometheus<br/>scrape every 5 s<br/>alert rules"]
        GRAF["Grafana<br/>Loan API dashboard"]
        DRIFT["Drift and fairness report<br/>PSI per feature<br/>offer ratio by group"]
    end

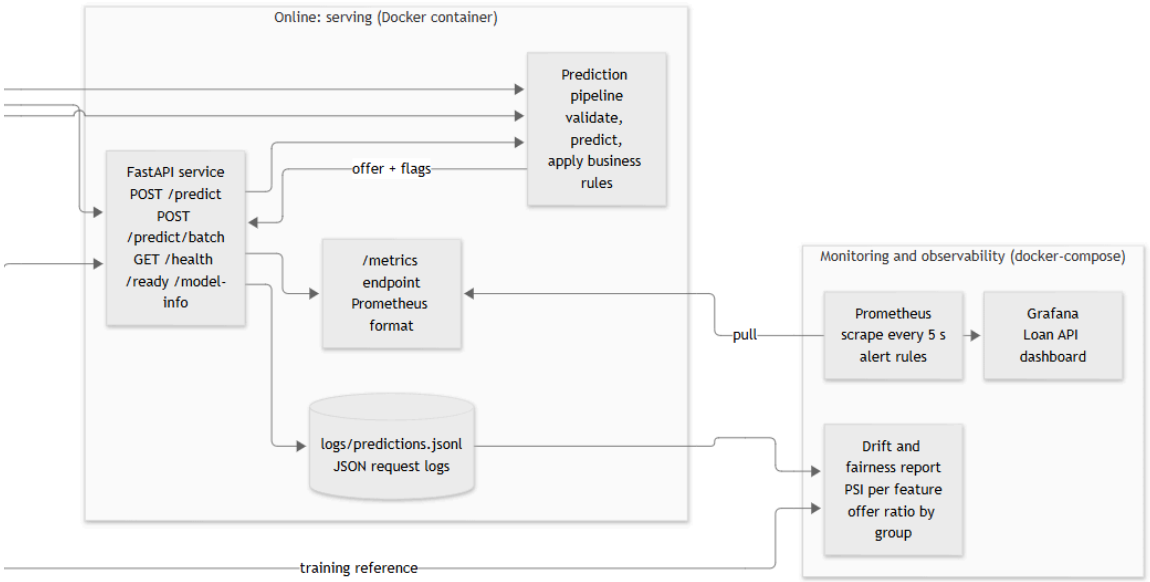
    CLIENT["Clients<br/>Swagger UI, curl,<br/>traffic simulator"]

    RAW --> PROFILE --> CLEAN --> FE --> TRAIN
    TRAIN --> PKL
    TRAIN --> META
    PKL --> PRED
    META --> API
    CLIENT -->|"JSON applicant"| API
    API --> PRED
    PRED -->|"offer + flags"| API
    API --> LOG
    API --> MET
    SAMPLE -->|"batch scoring"| PRED
```

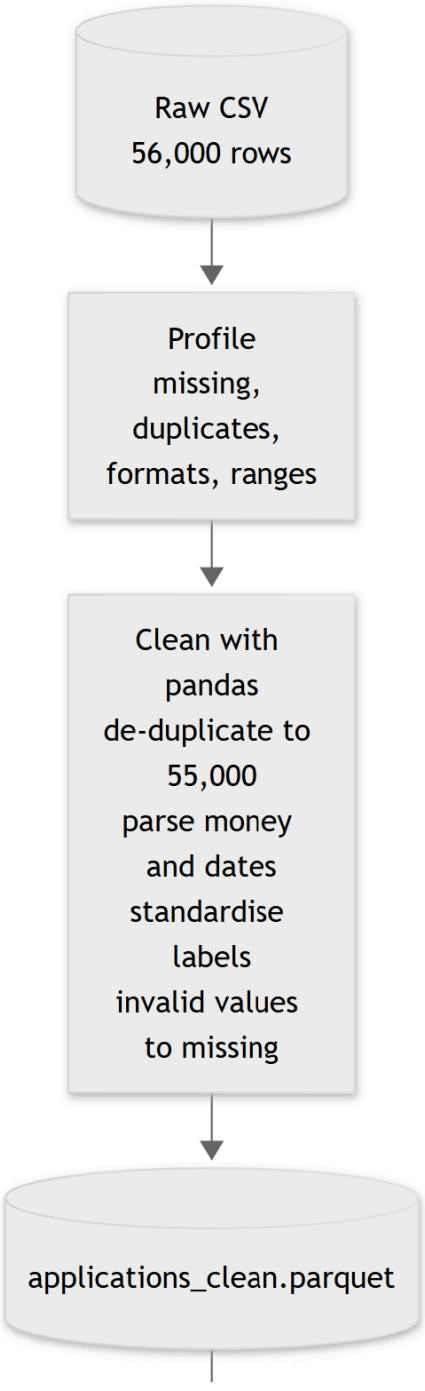
PROM -->|"pull"| MET
PROM --> GRAF
LOG --> DRIFT
META -->|"training reference"| DRIFT

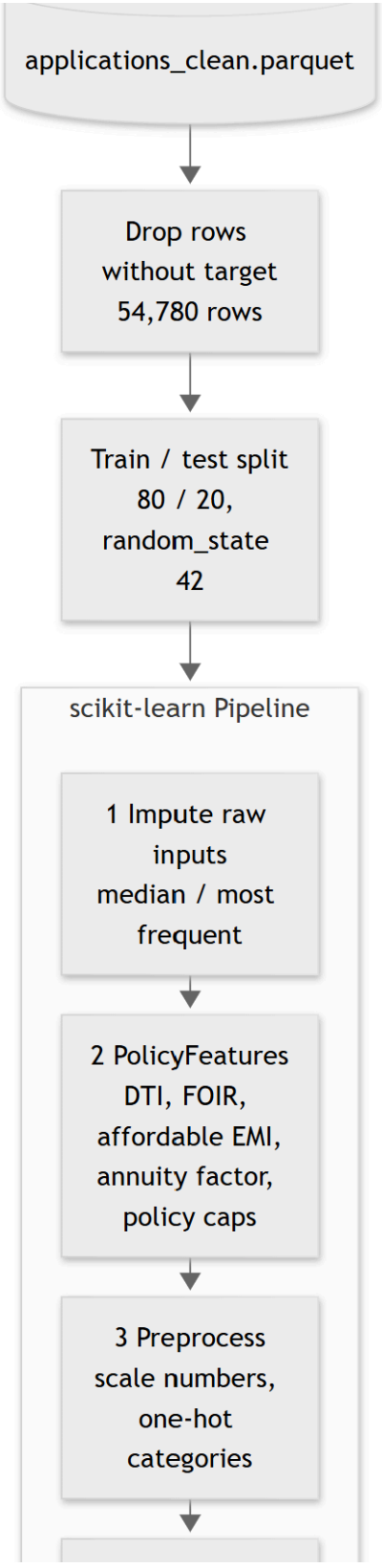
Part - 1

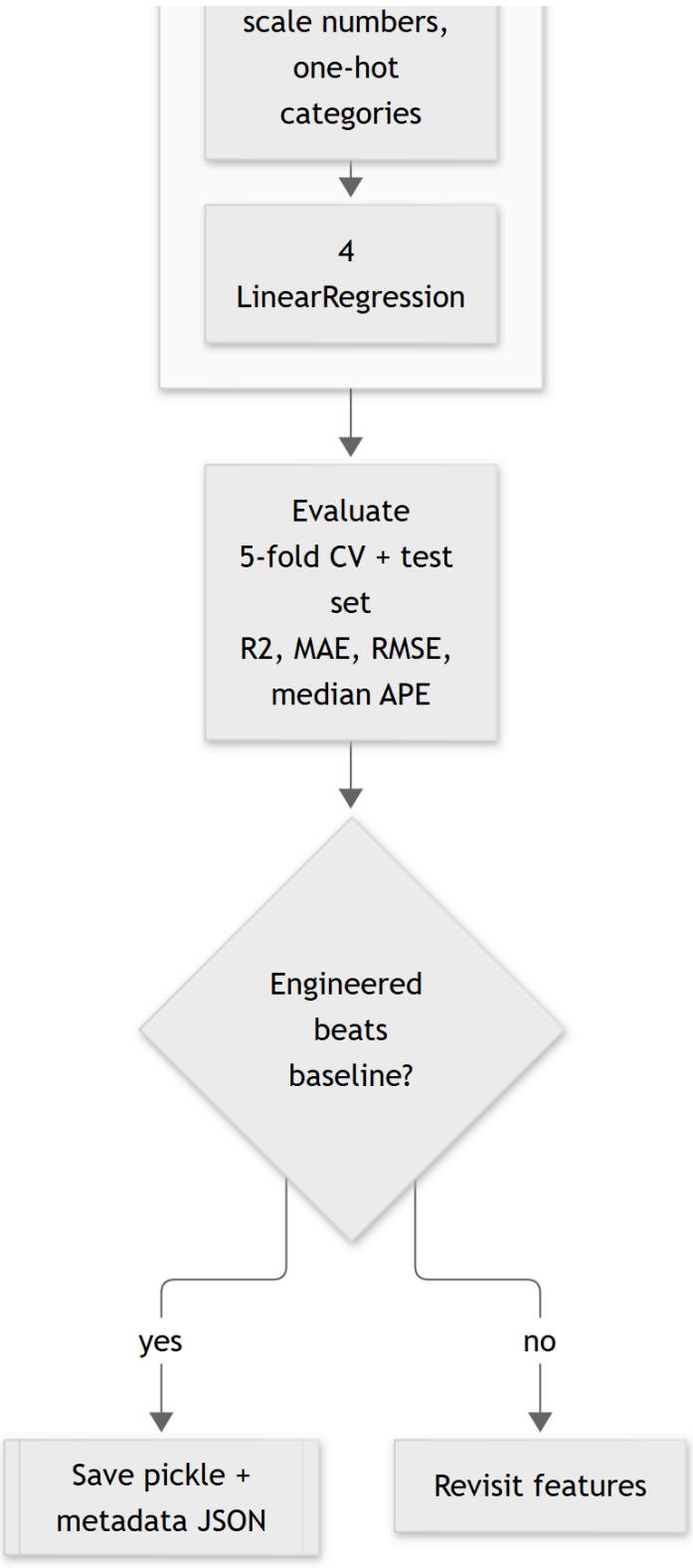




Training Pipeline




applications_clean.parquet
Drop rows without target 54,780 rows
Train / test split 80 / 20, random_state 42
scikit-learn Pipeline
1 Impute raw inputs median / most frequent
2 PolicyFeatures DTI, FOIR, affordable EMI, annuity factor, policy caps
3 Preprocess scale numbers, one-hot categories


scale numbers, one-hot categories
4 LinearRegression
Evaluate 5-fold CV + test set R2, MAE, RMSE, median APE
Engineered beats baseline?
yes
no
Save pickle + metadata JSON
Revisit features

```
flowchart TD
    A[("Raw CSV<br/>56,000 rows")] --> B["Profile<br/>missing, duplicates, formats, ranges"]
    B --> C["Clean with pandas<br/>de-duplicate to 55,000<br/>parse money and
dates<br/>standardise labels<br/>invalid values to missing"]
    C --> D[("applications_clean.parquet")]
    D --> E["Drop rows without target<br/>54,780 rows"]
    E --> F["Train / test split<br/>80 / 20, random_state 42"]
    F --> G

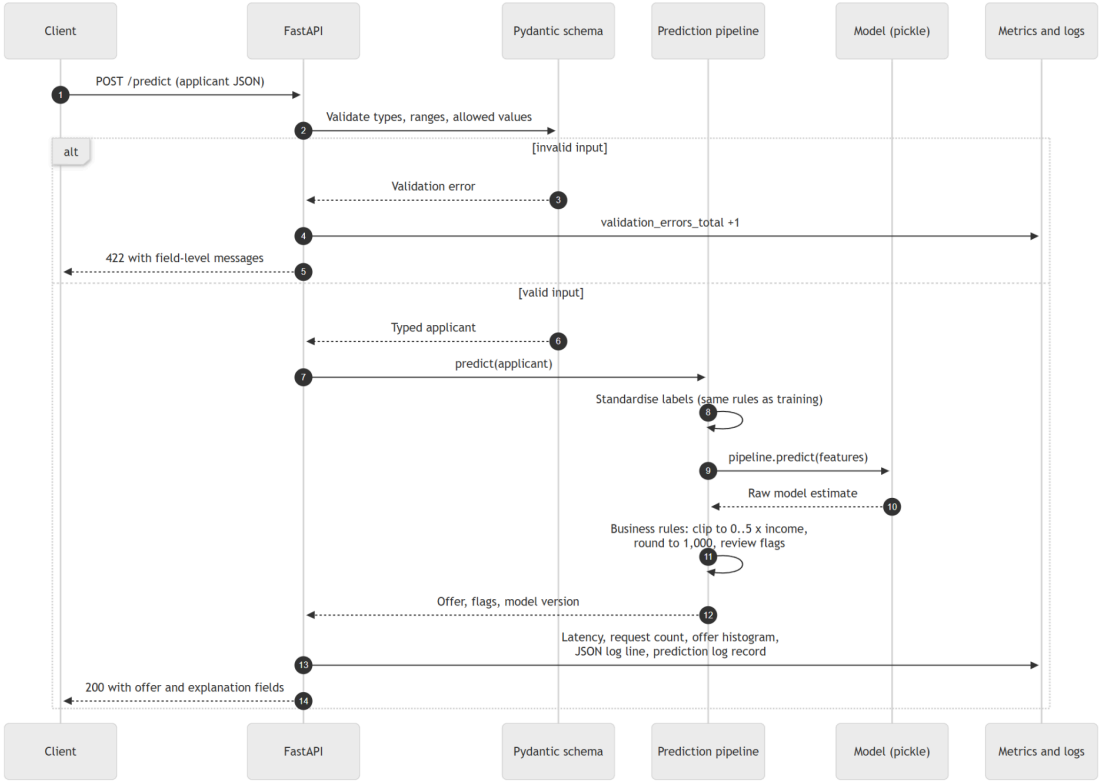
    subgraph G["scikit-learn Pipeline"]
        direction TB
        G1["1 Impute raw inputs<br/>median / most frequent"]
        G2["2 PolicyFeatures<br/>DTI, FOIR, affordable EMI,<br/>annuity factor, policy caps"]
        G3["3 Preprocess<br/>scale numbers, one-hot categories"]
        G4["4 LinearRegression"]
        G1 --> G2 --> G3 --> G4
    end

    G --> H["Evaluate<br/>5-fold CV + test set<br/>R2, MAE, RMSE, median APE"]
    H --> I{"Engineered beats<br/>baseline?"}
    I -->|"yes"| J[["Save pickle +<br/>metadata JSON"]]
    I -->|"no"| K["Revisit features"]
```

Sequence Diagram to Understand the flow

```
sequenceDiagram
    autonumber
    participant C as Client
    participant A as FastAPI
    participant V as Pydantic schema
    participant P as Prediction pipeline
    participant M as Model (pickle)
    participant O as Metrics and logs

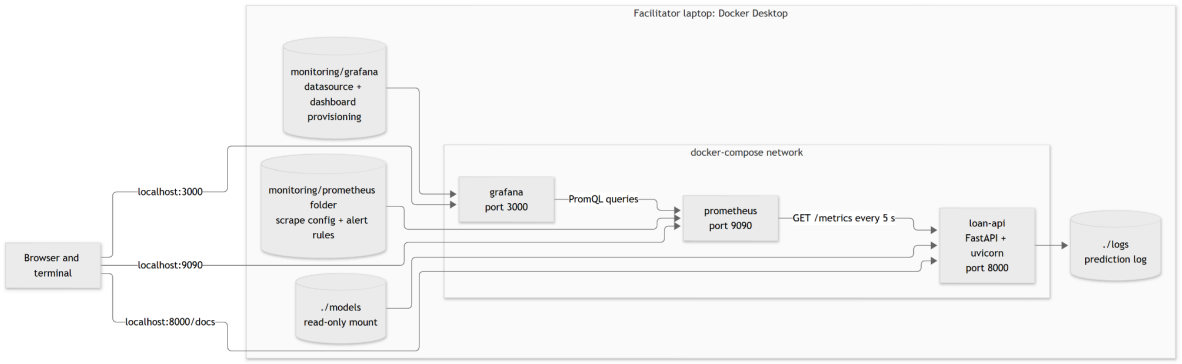
    C->>A: POST /predict (applicant JSON)
    A->>V: Validate types, ranges, allowed values
    alt invalid input
        V-->>A: Validation error
        A->>O: validation_errors_total +1
        A-->>C: 422 with field-level messages
    else valid input
        V-->>A: Typed applicant
        A->>P: predict(applicant)
        P->>P: Standardise labels (same rules as training)
        P->>M: pipeline.predict(features)
        M-->>P: Raw model estimate
        P->>P: Business rules: clip to 0..5 x income,<br/>round to
1,000, review flags
        P-->>A: Offer, flags, model version
        A->>O: Latency, request count, offer histogram,<br/>JSON log
line, prediction log record
        A-->>C: 200 with offer and explanation fields
    end
```



Business Rules applied after the model

- Offer is NEVER below 0
- Offer is NEVER above 5x ANNUAL INCOME
- Offer is ROUNDED DOWN to the nearest 1000s
- Manual Review = TRUE if credit score < 600 and debt-to-income > 0.6 or Credit Score is missing
- Response includes raw model estimate and rules applied

Architecture Diagram - Deployment of Application




```
flowchart LR
    subgraph HOST["Facilitator laptop: Docker Desktop"]
        subgraph NET["docker-compose network"]
            API["loan-api<br/>FastAPI + uvicorn<br/>port 8000"]
            PROM["prometheus<br/>port 9090"]
            GRAF["grafana<br/>port 3000"]
        end
        VOLM[("./models<br/>read-only mount")]
        VOLL[("./logs<br/>prediction log")]
        CFGP[("monitoring/prometheus folder<br/>scrape config + alert rules")]
        CFGG[("monitoring/grafana<br/>datasource + dashboard provisioning")]
    end
    USER["Browser and terminal"]

    USER -->|"localhost:8000/docs"| API
    USER -->|"localhost:9090"| PROM
    USER -->|"localhost:3000"| GRAF
    PROM -->|"GET /metrics every 5 s"| API
    GRAF -->|"PromQL queries"| PROM
    VOLM --> API
    API --> VOLL
    CFGP --> PROM
    CFGG --> GRAF
```

Case Study Dataset

> In the terminal window

> cd c:\demonstrations

> `git clone https://github.com/iomegak12/aiml-e2e`

> cd aiml-e2e

> code .

Technology Choices

Language - Python 3.12

Data Engineering - Pandas, NumPy and PyArrow (Parquet)

Modeling - SciKit-Learn

Statistics for Model Drifts - SciPy

REST API - FastAPI, Uvicorn

Request Validations - Pydantic Schemas

Metrics - Prometheus Client

Logging - Python JSON Logger

Testing - PyTest, HTTPX

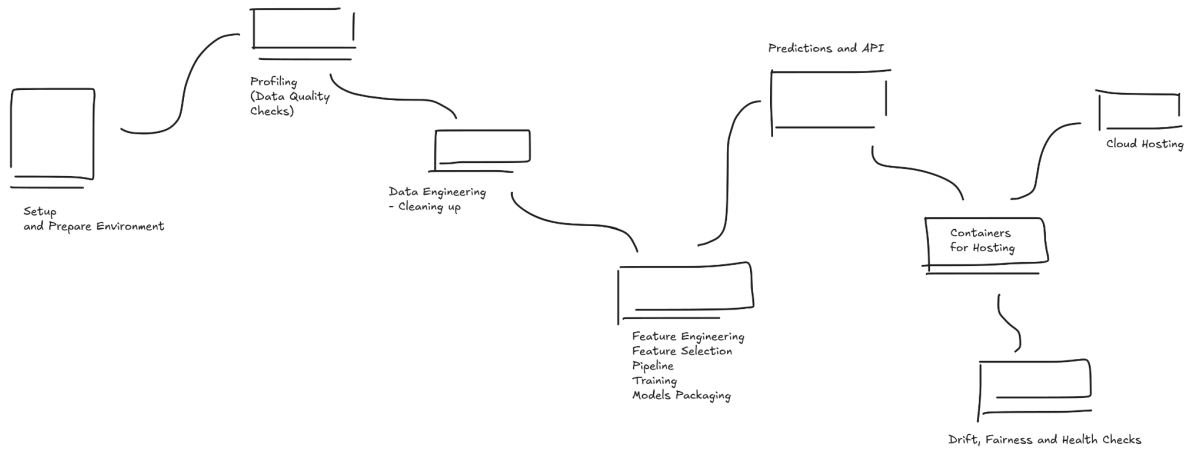
Containers - Docker Desktop and Docker Compose

Monitoring for Metrics - Prometheus

Dashboard for Observability - Grafana

Cloud Host - AWS EC2 Instances

Development Timeline and Workflow



Day 5 Agenda

-
- Implement the E2E Case Study - Final Chunks
    - Complete Reference Implementation
- Lecture on Classification Algorithms - 30 Minutes
- Describe the Business Use Case - Participants - Classification Algorithms
- Lecture on Generative AI, LLM
- Lecture on LangChain - Library for AI Application Development
- Demonstration & Practices - How to Build Agents with Tools and Other Features
- Agentic AI - Enterprise Features

Complete Source Code for the End-to-End Case Study Available at
https://github.com/iomegak12/aiml-e2e

Classification Algorithms & Metrics

When a program learns to put things into groups, we call it classification.

- Binary Classification - Yes / No
- Decision Boundary
- Threshold
- Logistic Regression
    - It exactly does the job of adding score for every characteristic (feature) - clue a weight - How many points it's worth - Then add them all up into one score
    - Squeeze Score into Range
    - Sigmoid Curve - Turns into 0 to 1
    - Odds Ratio
- How does it learn?
    - Try, Measure the mistake, adjust and repeat
    - Confident mistakes are punished more harshly than unsure ones.
    - Log Loss
    - Cost Function
    - It's the model that measures how wrong it is.
    - Gradient Descent
    - Overfitting
    - Regularization
    - Mutl-class classification - The trick is to train three or more separate yes-or-no models
    - One vs. Rest
- K-Nearest Neighbors (KNN)
    - Lazy Learner
    - How do we measure the similarity?
        - Euclidean Distance
        - Manhattan Distance
    - How many neighbors do we work or ask?
        - K - is the number of neighbors in KNN
- K-Means
    - Its job is to find natural groups on its own
    - No prior examples ARE needed
    - Unsupervised Learning
    - In KNN, it's the number of neighbors, but when it comes to K-Means, it's the number of groups you want.
    - How does it work?
        - Moving the average to the center point of the group
        - Centroid
        - How do you choose the K?
            - Yourself based on your business requirements
        - Elbow Method
        - Silhouette Method

- Metrics
    - Accuracy
    - Precision
    - Recall
    - F1 Score

- For example, in Email Spam Filter scenarios,
    - 1000 Emails Arrived
    - 100 of them were actually spam
    - The filter caught 80 of them spam emails
    - It missed 20 spam emails, they landed in the inbox
    - It also wrongly sent 30 good emails to the spam folder
    - The other 870 emails were good, correctly left in the inbox

- Spam and Caught - True Positive
- Spam and Missed - False Negative
- Good Email and wrongly went to the spam - False Positive
- Good Email and Left along - True Negative

- If they're fit into 2x2 table, Confusion Matrix
- Accuracy
    - When one group is much bigger than the other, accuracy can fool you.
- Precision
    - When it raises the alarm, can I trust it?
- Recall
    - Did it catch most of what it should be catching?
- F1 Score
    - F1 score does that in a strict way of combining both precision and recall together

Other Important Metrics

- ROC (Receiver Operating Characteristic)
- AUC (Area Under Curve)

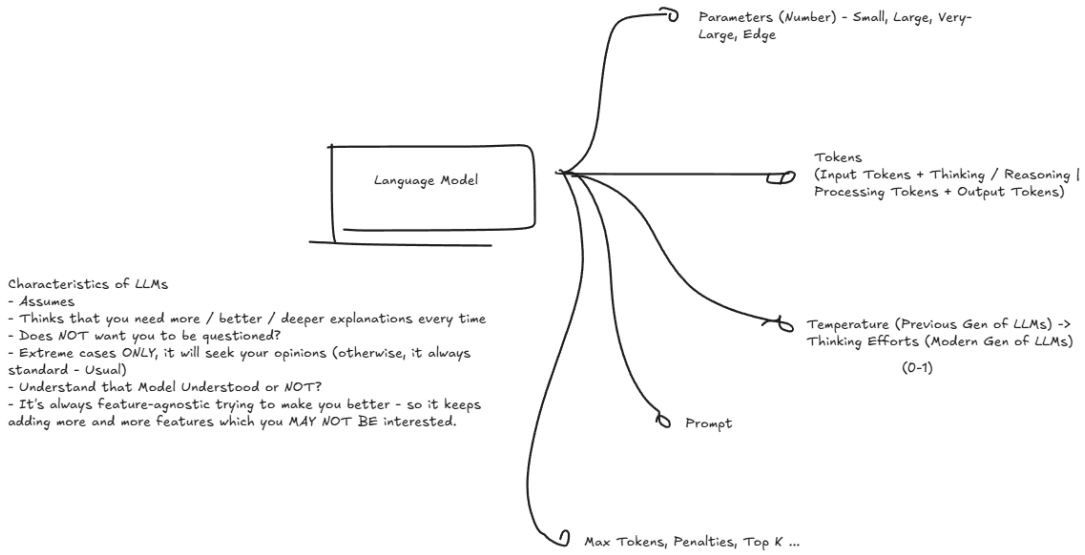
NOTE: Students specific Business Use Case - Sample Data, Real Dataset and Business Use Case Expectations and Utility Scripts to generate a new raw data are available at https://github.com/iomegak12/aiml-students-e2e

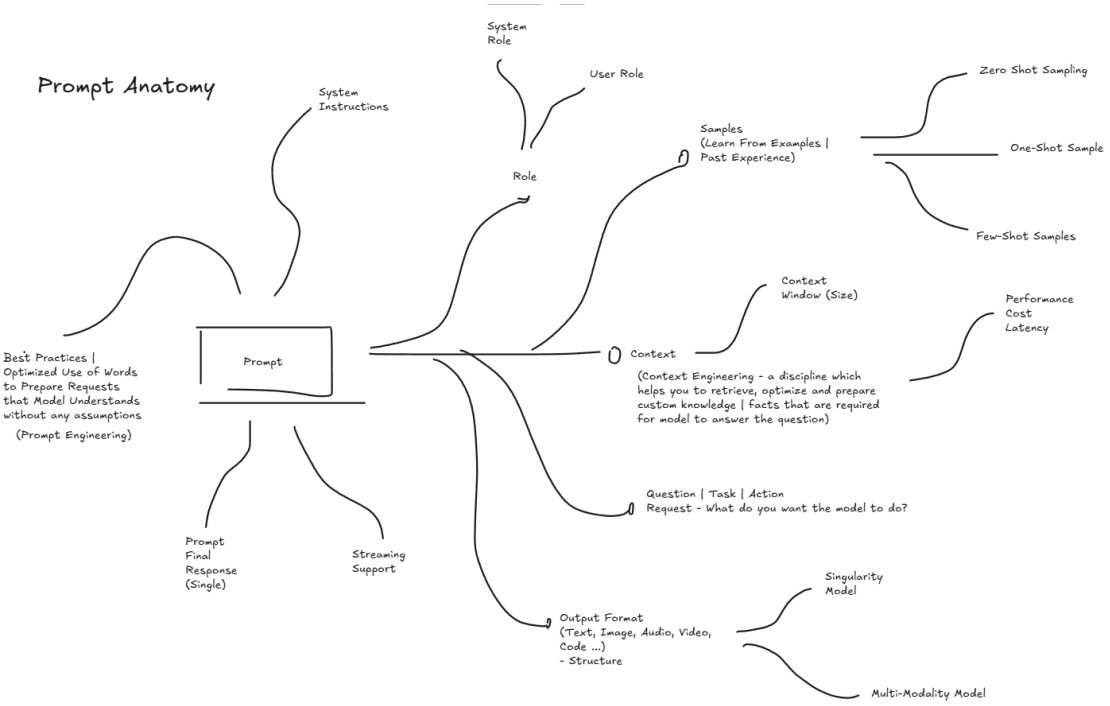
Generative AI, LLMs - Agentic AI

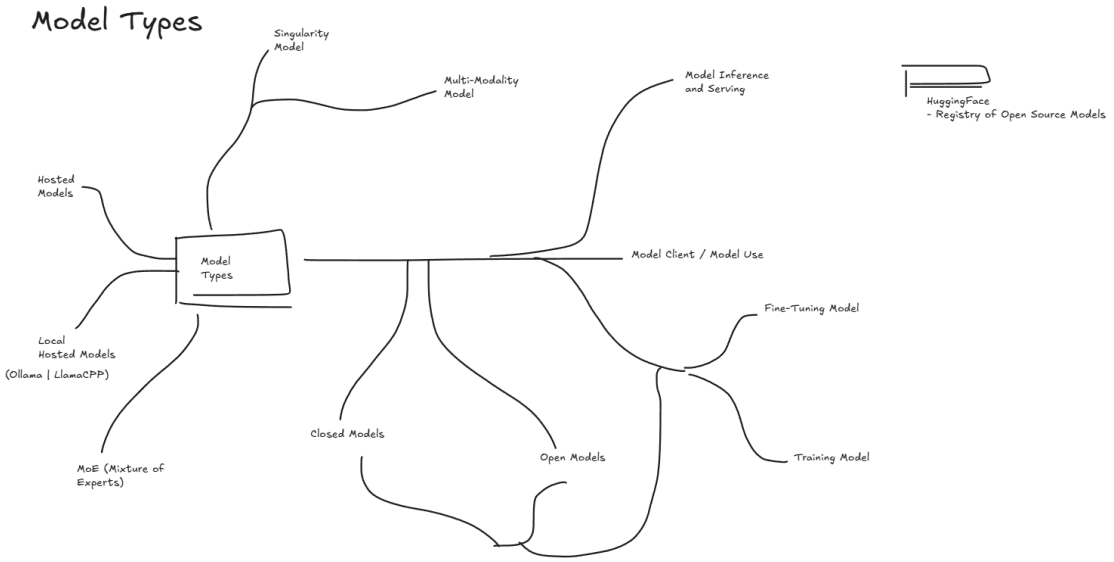
Generative AI

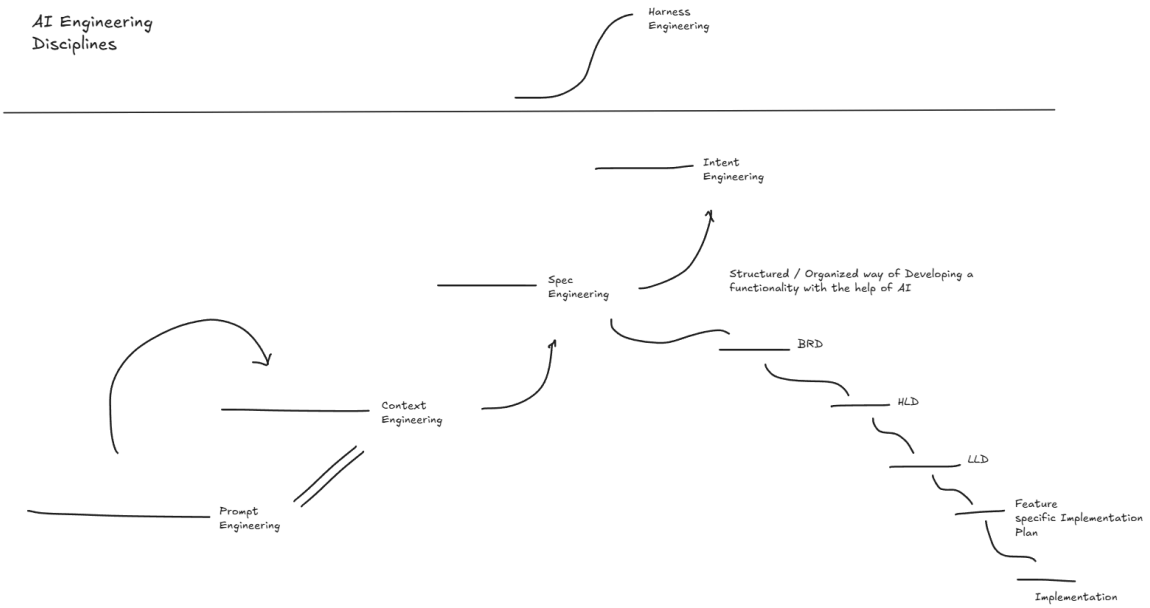
- Advancement of DL - DNNs - a DL model that has the ability to generate new contents based on the training data, like how humans understand and respond - with NL* capabilities
    - NL* (Natural Language)
        - NLU (Understanding
        - NLP (Processing)
        - NLR (Response)

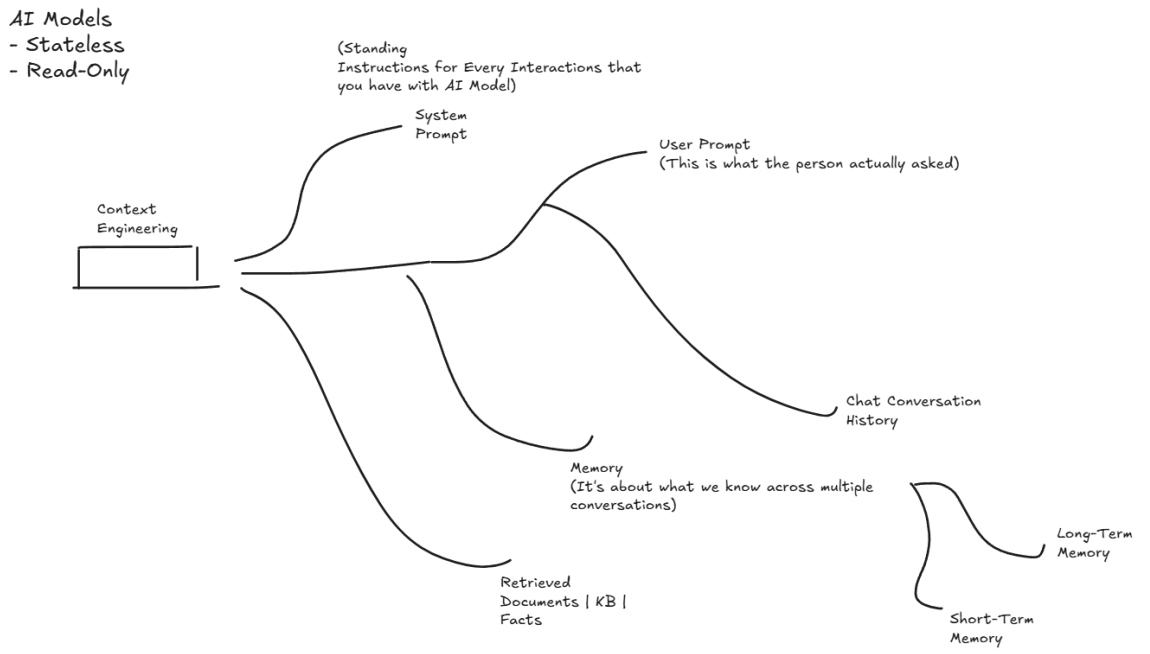
- LM (Language Model)
    - Model (DL Model) - Language (NL*)

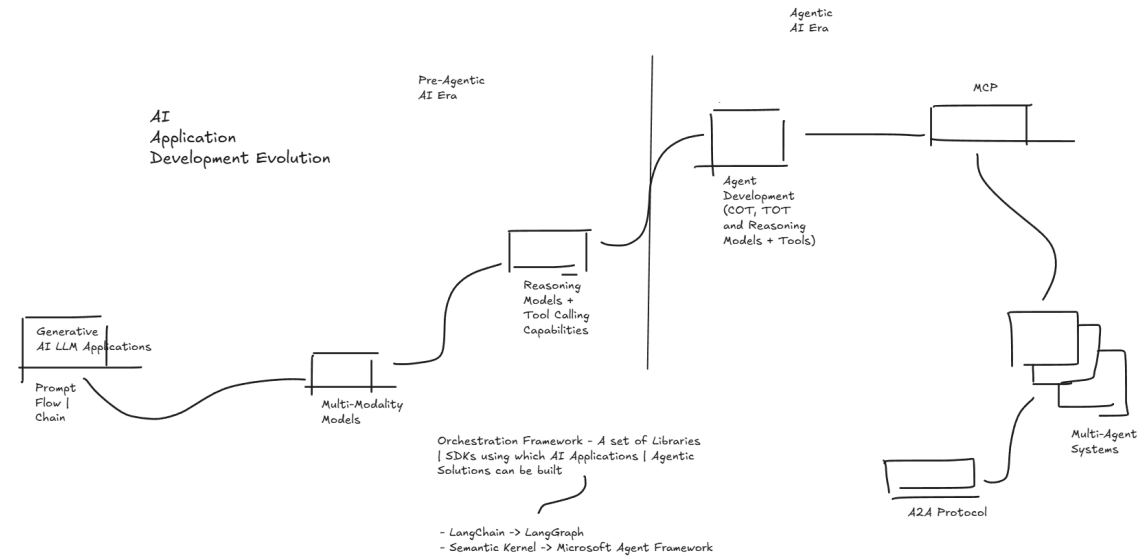



Prompt Anatomy
System Role
User Role
System Instructions
Role
Samples (Learn From Examples | Past Experience)
Zero Shot Sampling
One-Shot Sample
Few-Shot Samples
Context Window (Size)
Performance
Cost
Latency
Best Practices | Optimized Use of Words to Prepare Requests that Model Understands without any assumptions
(Prompt Engineering)
Prompt
Context
(Context Engineering - a discipline which helps you to retrieve, optimize and prepare custom knowledge | facts that are required for model to answer the question)
Question | Task | Action Request - What do you want the model to do?
Prompt Final Response (Single)
Streaming Support
Singularity Model
Output Format (Text, Image, Audio, Video, Code ...) - Structure
Multi-Modality Model


Model Types
Singularity Model
Multi-Modality Model
Model Inference and Serving
HuggingFace - Registry of Open Source Models
Hosted Models
Model Types
Model Client / Model Use
Fine-Tuning Model
Local Hosted Models
(Ollama | LlamaCPP)
Closed Models
Open Models
Training Model
MoE (Mixture of Experts)


AI Engineering
Disciplines
Harness
Engineering
Intent
Engineering
Spec
Engineering
Structured / Organized way of Developing a
functionality with the help of AI
BRD
HLD
LLD
Feature
specific Implementation
Plan
Implementation
Context
Engineering
Prompt
Engineering


AI Models
- Stateless
- Read-Only
(Standing
Instructions for Every Interactions that
you have with AI Model)
System
Prompt
User Prompt
(This is what the person actually asked)
Context
Engineering
Chat Conversation
History
Memory
(It's about what we know across multiple
conversations)
Long-Term
Memory
Short-Term
Memory
Retrieved
Documents | KB |
Facts


AI
Application
Development Evolution
Pre-Agentic
AI Era
Agentic
AI Era
MCP
Generative
AI LLM Applications
Prompt
Flow |
Chain
Multi-Modality
Models
Reasoning
Models +
Tool Calling
Capabilities
Agent
Development
(COT, TOT
and Reasoning
Models + Tools)
Multi-Agent
Systems
A2A Protocol
Orchestration Framework - A set of Libraries
| SDKs using which AI Applications | Agentic
Solutions can be built
- LangChain -> LangGraph
- Semantic Kernel -> Microsoft Agent Framework

Agentic AI Demonstration & Practices

> in terminal window ->

> cd c:\demonstrations

> git clone https://github.com/iomegak12/aiml-agentic-ai-1

> cd aiml-agentic-ai-1

> uv sync

> code .

> copy & paste the .env.example file

> rename it to .env

> paste the following OpenAI API Key

```
OPENAI_API_KEY=sk-proj-bwrFA7rGvkCx63rmhOkVBFOti46zoAPxDwDOfFTcDTQAlwzn
_ixL3ta5o-Ci28mj_WGbHfjbj9T3BlbkFJuSG6SbScjMjMIACg8Ihq8zjBd4LjoMwTKgihk
2lKVIotqrn8RDmNVAn-jVjIUSWAUZ_Gg3W10A
```

> open the notebook in notebooks\it-support-agent.ipynb

> view -> command palette -> select python interpreter -> virtual environments -> select workspace specific python virtual environment

> select the kernel

> make sure that you select VENV python kernel which belongs to the current workspace

> execute the cell one by one

Day 6 Agenda

- Summarize Day 1 to Day 5 Contents
- Lecture Blocks
    - Lecture on RAG (Retrieval Augmented Generation)
        - Embedding Models
        - Vector Databases
        - RAG - Types of RAG
    - Lecture on Information Retrieval
        - Chunking & Indexing Techniques
        - Optimization Techniques
    - Getting things done with Agents
        - Workflow Concepts
        - Multi-Agent Systems
        - MCP
        - LangGraph
- Demonstration & Practices
    - RAG
    - Optimization
    - Agentic AI with MCP / Workflows

3 Blocks

- Finding the Right Information
- Making the Retrieval Work Well
- Getting Things Done with Agents

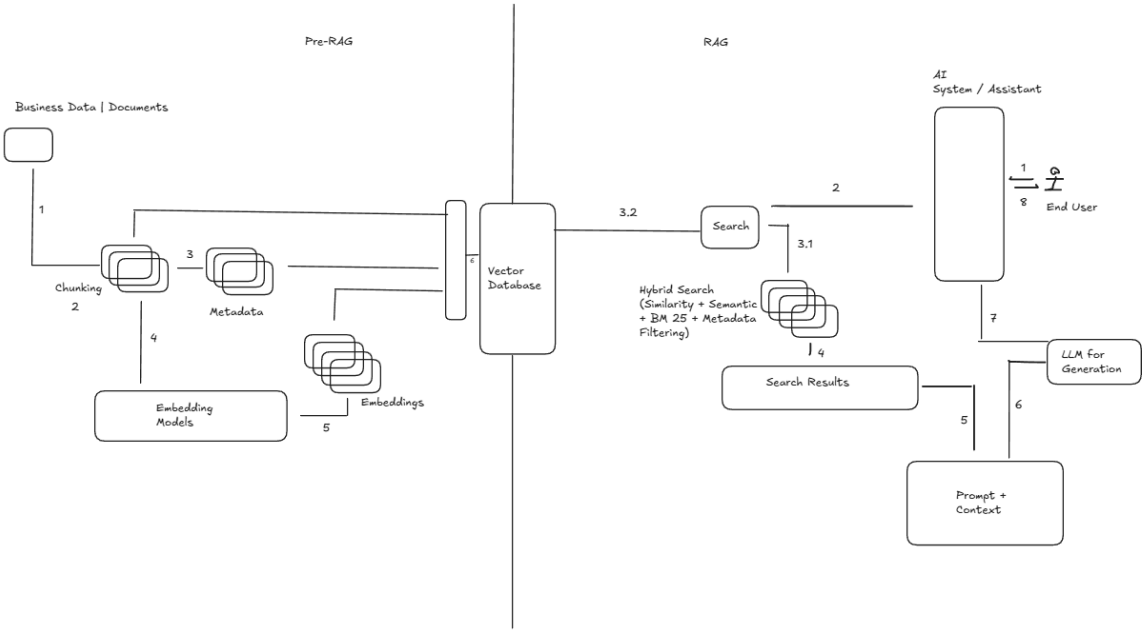
In simple terms, RAG means,

Teaching an AI assistant to take an Open-Book Exam. To do that, we need three things:

- A way to understand what each page is about (Embeddings)
- A place to keep all those pages so we can find the right ones in a split second (Vector Databases)
- A method that finds the right pages, and hands them to the assistant before it answers (RAG)

Important Terminologies

- Embedding
- Vector
- Embeddings
- Vectors (Vectorization)
- Embedding Models
    - Dimensions
        - 512 Dimensions (Minimum)
        - 3500+ Dimensions
    - Open Source Embedding Models
    - Closed Source Embedding Models
        - Model Providers give you access to Closed Source Embedding Models
        - Open AI - text-embedding-3-small, text-embedding-3-large
- Embedding Database | Vector Database
- Chunking
- Indexing
- Metadata
- Document (Embeddings + Metadata + Original Text)
- Similarity Search
- Cosine Similarity Search
- Vector Search
- Keyword | Term Search
    - Exact Match
    - BM25 (Best Match 25)
- Hybrid Search (Similarity Search + Metadata Filtering + BM25 Search)
- Semantic Search
    - Meaningful Search with the context



We need an orchestration framework | set of Libraries that specialize providing the above capabilities to simplify, improve productivity and proven techniques to solve enterprise RAG challenges.

- LangChain
- Llama Index
- Microsoft Agent Framework

Common Vector Databases

- RDBMS compatible Vector Databases
    - SQL Server
    - Postgres + PGVector
    - Oracle
- NoSQL compatible Vector Databases
    - Mongo Database
    - Elastic Search
- Native Vector Databases
    - Small Scale
        - Chroma
        - FAISS
    - SaaS
        - Pinecone
    - Open Source - Enterprise Ready
        - Qdrant
        - Milvus
        - Weaviate

RAG

- Retrieval - Go and Fetch the Relevant Information
- Augmented - Add it to the context, so the assistant knowledge is ENRICHED
- Generation - The LLM writes the answer using the fetched document

The family of RAG Types

- 9 Commonly used RAG Techniques | Types - Most of them are specific to business use cases
- Basic RAG (Naive RAG)
- Advanced RAG
- Hybrid RAG
- Conversational RAG
- Multi-Query RAG
- Corrective RAG
- Agentic RAG
- Graph RAG
- Multi-Modal RAG

Basic RAG - Naive RAG

- Embed the question, fetch the top few chunks and answer the question based on the chunks.
- Fast, Simple and Often good enough

Advanced RAG

- Before searching, it might tidy up or rephrase the question - After searching, it double checks which of the found chunks are TRULY the most relevant, and puts them in the best order - prepare the context and answer them
- Before the answer is given to the end user, evaluate with the another Judge LLM
    - Question
    - Context
    - Answer

Hybrid RAG

- This searches in three different ways at ONCE
    - Similarity Search
    - Semantic Search
    - Exact Keyword | Phrase match (BM25 Search)
- Combines the Results

Conversational RAG

- It uses the chat history to rewrite the follow-up into a complete question.

Multi-Query RAG

- A user asked question is divided into multiple sub-questions. And each question is separately searched for results, combine / summarize the results before the question is answered by LLM
- This type of RAG asks LLM to break that into smaller questions, searches for each one separately and then combines everything

Corrective RAG

- Basic RAG trusts whatever the search returns.
- But corrective RAG adds a checker.
- After retrieving, it asks “are these chunks relevant actually to the user questions?” - If it’s NOT matching, rephrases the question, searches a different source and admits if it does NOT know. After writing, it can also check, “Is every statement in my answer supported by the sources?”
- Enterprise Ready RAG

Agentic RAG

- An agent, which decides whether to search at all, where to search, where to search again. It might check the policy library, then look up the customers’ account in the CRM, then search the support history, and then answer.
- Most effective, time-consuming, cost bounded and importantly slowest

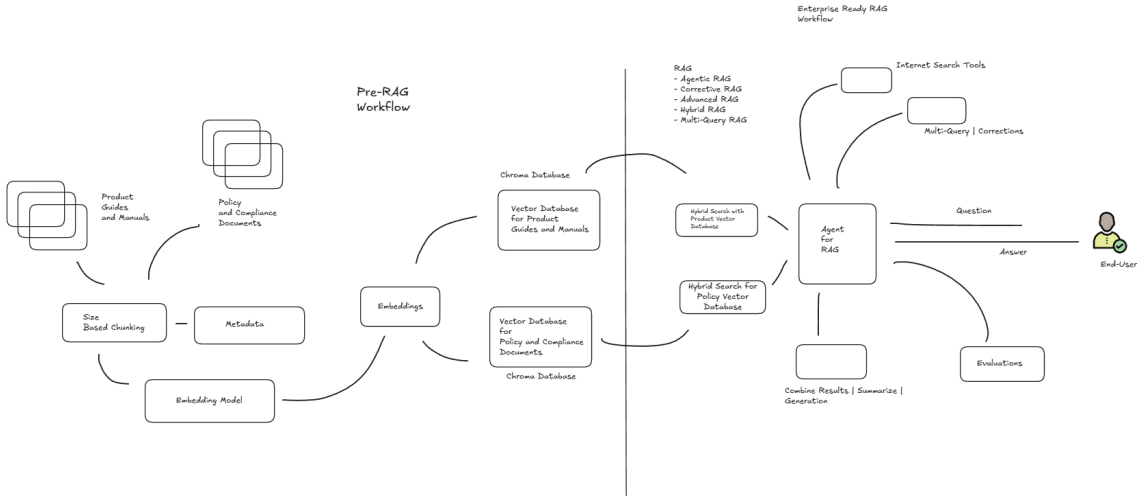
Graph RAG

- It builds a knowledge graph, out of all available data sources, a web of things (customers, products and partners) and the relationships between them (uses, manages and renewed with). Then it answers by following the links

Multi-Modal RAG

- It can retrieve and understand those NON-TEXT documents also, including Audio, Video, Images, Screenshots, Diagrams, Scanned Images out of Handwritten documents.
- Multi-Modality Models - understand Non-Text documents, prepare a descriptive text out of those documents and embeddings are then stored in the vector database.

Case Study - RAG



prompt-1 = data generation
prompt-2 = pre-rag activities
prompt-3 = RAG activities
prompt-4 = MCP Server development
prompt-5 = Multi-Agent Systems Development with LangGraph

Day 6 Case Study Implementation available at
https://github.com/iomegak12/aiml-day6